TUGAS BESAR I
IF3270 PEMBELAJARAN MESIN

IMPLEMENTASI FEEDFORWARD NEURAL NETWORK DARI *SCRATCH* DAN PENERAPANNYA DI KLASIFIKASI BINER

Disusun oleh:

Kelompok 67
Six-seven

Albi Arrizkya Putra (10122062)

Timothy Niels Ruslim (10123053)

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

I DESKRIPSI TUGAS **1**
1.1. Implementasi FFNN 1
1.2. Analisis Dataset 2

II PEMBAHASAN **3**
2.1. Implementasi FFNN 3
2.1.1. `engine.py` 3
2.1.2. `activation.py` 6
2.1.3. `initialize.py` 7
2.1.4. `loss.py` 7
2.1.5. `optimizer.py` 8
2.1.6. `nn.py` 9
2.2. Hasil Pengujian 14
2.2.1. Pengaruh *Depth* dan *Width* 14
2.2.2. Pengaruh Fungsi Aktivasi 19
2.2.3. Pengaruh *Learning Rate* 26
2.2.4. Pengaruh Inisialisasi Bobot 31
2.2.5. Pengaruh Regularisasi 36
2.2.6. Pengaruh Normalisasi RMSNorm 41
2.2.7. Perbandingan dengan *Library* Sklearn 41
2.2.8. Perbandingan *Optimizer* SGD vs ADAM 42

III KESIMPULAN DAN SARAN **44**
3.1. Kesimpulan 44
3.2. Saran 45
3.3. Lampiran Tautan 46
3.4. Lampiran Pembagian Tugas 46

BAB I
DESKRIPSI TUGAS

Tugas Besar I mata kuliah IF3270 Pembelajaran Mesin ini bertujuan untuk mengenali kami pada cara implementasi dan penerapan *Feedforward Neural Network* (FFNN). Khususnya, kami ditugaskan untuk mengimplementasikan *FFNN from scratch*, lalu menggunakan implementasi tersebut untuk melakukan analisis dan pemodelan terhadap dataset *Global Student Placement & Salary Dataset*.

1.1. Implementasi FFNN

Modul FFNN yang diimplementasikan harus memenuhi ketentuan-ketentuan berikut.

- **Width and Depth**. FFNN dapat menerima jumlah neuron dari tiap *layer*, termasuk *input layer* dan *output layer*.
- **Activation**. FFNN dapat menerima fungsi aktivasi dari tiap *layer*. Beberapa pilihan fungsi aktivasi yang harus diimplementasikan adalah: Linear, ReLU, Sigmoid, Tanh, dan Softmax. Di sini, kami juga mengimplementasikan Swish dan GeLU.
- **Loss**. FFNN dapat menerima fungsi *loss* dari model tersebut. Beberapa pilihan *loss function* yang harus diimplementasikan adalah: mean-squared error, binary cross-entropy, dan categorical cross-entropy.
- **Initialization**. Terdapat mekanisme untuk inisialisasi bobot tiap neuron. Pilihan metode inisialisasi bobot yang harus diimplementasikan adalah: zero initialization, random dengan distribusi $\mathcal{U}(a, b)$, dan random dengan distribusi $\mathcal{N}(\mu, \sigma^2)$. Di sini, kami juga mengimplementasikan Xavier dan He initialization.
- **Weights and Gradient**. Instance model yang diinisialisasikan harus bisa menyimpan bobot tiap neuron, menyimpan gradien bobot tiap neuron, menampilkan distribusi bobot dari tiap *layer*, menampilkan distribusi gradien bobot dari tiap layer, dengan menerima masukan berupa *list of integer* (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan *layer* mana saja yang distribusinya akan di-plot.
- **Storage**. Instance model memiliki method untuk *save* dan *load.*
- **Forward and Backward Propagation** Model memiliki implementasi *forward propagation* dengan menerima input berupa *batch* dan memiliki implementasi *backward propagation* untuk menghitung perubahan gradien. Proses *backward propagation* dapat menangani perhitungan perubahan gradien untuk input data *batch*, menggunakan konsep *chain rule* untuk menghitung gradien tiap bobot terhadap *loss function.*
- **Regularization**. Model memiliki implementasi metode regularisasi L1 dan L2.
- **Gradient Descent**. Model memiliki implementasi *weight update* dengan menggunakan *gradient descent* untuk memperbarui bobot berdasarkan gradien yang telah dihitung.

- **Training**. Implementasi untuk pelatihan model dapat menerima parameter Batch Size, Learning Rate, Jumlah Epoch, dan Verbose. Verbose 0 berarti tidak menampilkan apa-apa selama pelatihan. Verbose 1 berarti hanya menampilkan progress bar beserta dengan kondisi training loss dan validation loss saat itu. Proses pelatihan mengembalikan histori dari proses pelatihan yang berisi training loss dan validation loss tiap epoch.

1.2. Analisis Dataset

Berikutnya, perlu dilakukan analisis dan pemodelan menggunakan dataset *Global Student Placement & Salary Dataset* menggunakan FFNN yang sudah dibuat.

- **Dataset**. Dataset yang digunakan adalah *Global Student Placement & Salary Dataset* dari `datasetml_2026.csv`, yang berisi kumpulan data global mengenai performa akademik mahasiswa serta outcome penempatan kerja dan gaji setelah lulus Target adalah kolom `placement_status`, sedangkan kolom lainnya dapat menjadi fitur.

- **Hyperparameter**. Analisis pengaruh beberapa hyperparameter sebagai berikut.

  - *Pengaruh arsitektur*. Pengaruh depth (banyak layer) dan width (banyak neuron per layer). Pilih 3 variasi kombinasi width (depth tetap) dan 3 variasi depth (width semua layer tetap). Bandingkan hasil akhir prediksinya. Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
  - *Pengaruh fungsi aktivasi hidden layer*. Tentukan base arsitektur (minimal 3 layer), pilih 1 layer sebagai layer pengetesan. Lakukan eksperimen dengan menjalankan variasi base arsitektur dengan setiap fungsi aktivasi yang diimplementasikan kecuali softmax pada layer pilihan. Bandingkan hasil akhir prediksinya. Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan. Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
  - *Pengaruh learning rate*. Lakukan 3 variasi learning rate (nilainya dibebaskan). Bandingkan hasil akhir prediksinya. Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan. Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.

- **Regularization**. Lakukan eksperimen masing-masing 1 kali untuk model tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2. Bandingkan hasil akhir prediksinya. Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan. Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.

- **Comparison**. Analisis perbandingan hasil prediksi dengan fungsi MLP di library `sklearn`. Lakukan satu kali pelatihan dengan hyperparameter yang sama untuk kedua model. Hyperparameter yang digunakan dibebaskan. Bandingkan hasil akhir prediksinya saja.

BAB II
PEMBAHASAN

2.1. Implementasi FFNN

Untuk membangun FFNN *from scratch*, kami membuat package `ffnn` yang melakukan semua implementasi sesuai spesifikasi di atas. Kami melakukan modularisasi kode sebagai berikut.

```
ffnn/
├── activation.py
├── engine.py
├── initialize.py
├── loss.py
├── nn.py
└── optimizer.py
```

Penjelasan selebihnya dilakukan satu-satu di subbagian-subbagian berikut.

2.1.1. `engine.py`

File `engine.py` bertujuan untuk menampung kelas `Tensor` yang menjadi struktur data matematis utama untuk FFNN. Secara khusus, berikut kelas-kelas yang dibuat.

Tabel 1: Daftar Kelas pada `engine.py`

Kelas	Atribut	Deskripsi
`GradMode`	`enabled`	Hanya untuk menyimpan mode perhitungan gradien.
`no_grad`	`prev`	Hanya untuk toggle antar mode gradien.
`Tensor`	`data`, `grad`, `_children`, `_backward`	Objek yang merepresentasikan barang matematis (angka, matriks, etc.) yang dapat dioperasikan. Strukturnya seperti simpul pada graf, sehingga memiliki simpul anak `_children`, dan dilengkapi fungsi `_backward` untuk bisa menghitung gradien suatu fungsi terhadap `Tensor` tersebut menggunakan nilai gradien anak-anaknya.

Lalu, ada juga satu fungsi pembantu untuk kelas `Tensor` (dan sebenarnya `ndarray` seumumnya), yaitu untuk melakukan *unbroadcasting* yang dilakukan secara otomatis oleh `numpy`.

Tabel 2: Fungsi Global `engine.py`

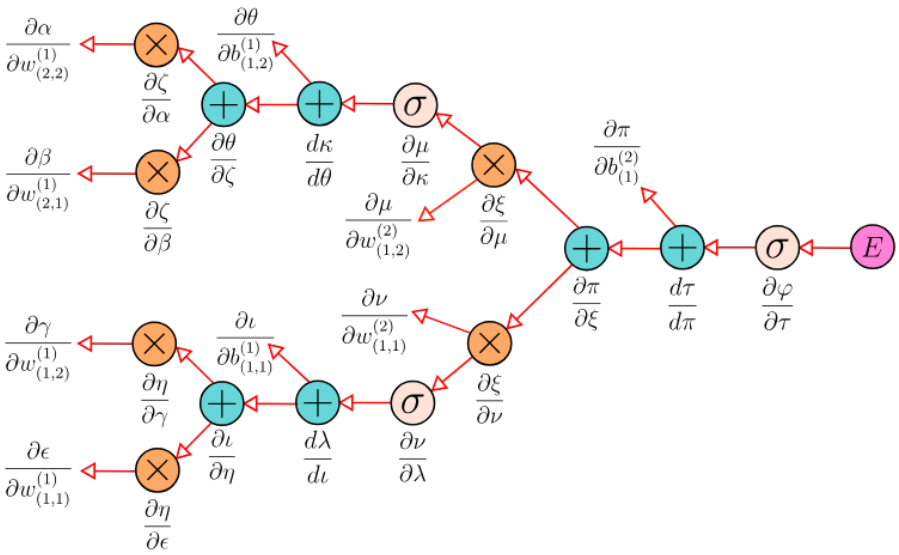
Metode	Parameter	Return	Deskripsi
`unbroadcast`	`grad` (array), `shape` (tuple)	`ndarray`	Melakukan *reverse broadcasting* dengan menjumlahkan nilai gradien pada axis yang diperluas untuk mendaptakan dimensi `Tensor` semula.

Kelas *Tensor* sendiri menjadi alasan implementasi *forward* dan *backpropagation* menjadi lebih mudah. Selain menampung nilai matematis, dia berperan sebagai simpul dalam graf komputasi *automatic differentiation*, mengikuti tutorial [4]. Secara ringkas, berikut mekanisme bekerjanya suatu `Tensor`.

- Misalkan dimiliki beberapa tensor $T_1, T_2, \ldots, T_n$. Lalu, kita menerapkan suatu operasi f pada tensor tersebut, yakni kita menghitung $T = f(T_1, T_2, \ldots, T_n)$.
- Pertama, dalam pembentukan T, dicatat bahwa $\{T_1, T_2, \ldots, T_n\}$ adalah "anak-anak" dari T.
- Kedua, dicatat jika kami mengetahui nilai $\partial\mathcal{L}/\partial T$ untuk suatu fungsi $\mathcal{L}$ dari T, bagaimana bisa mendapatkan nilai turunan parsial anak-anaknya, atau $\partial\mathcal{L}/\partial T_i$. Yakni, kita mencatat aturan rantai:

$$\frac{\partial\mathcal{L}}{\partial T_i} = \frac{\partial\mathcal{L}}{\partial T} \cdot \frac{\partial f}{\partial T_i}.$$

Secara komputasional di kode, ini berarti kita punya metode `_backward` untuk operasi f, yang output-nya hanyalah nilai $\partial\mathcal{L}/\partial T_i$ tersebut. Ini akan menghasilkan suatu graf komputasi besar (yang strukurnya sendiri sudah menyerupai MLP sebenarnya) yang bisa di-*traverse* dari simpul daun ke simpul akar saat melakukan *forward pass*, dan sebaliknya dari simpul akar ke simpul daun saat melakukan *backpropagation*.



Gambar 1: Graf Komputasi *Automatic Differentiation* [5]

Yang sangat menarik dari implementasi di atas ini, adalah bahwa kita dapat mendefinisikan hanya operasi-operasi umum pada `Tensor`, seperti penjumlahan, negatif, perkalian, dan perpangkatan. Lalu, semua operasi kompleks lainnya, seperti perhitungan suatu fungsi *loss*, atau suatu aktivasi non-linear σ, bisa didefinisikan hanya menggunakan operasi-operasi dasar tersebut. Dengan melakukan itu, perhitungan turunan simpul anak bisa dilakukan secara bertahap oleh operasi dasar tersebut, sehingga kami tidak perlu merepotkan apa itu turunan dari fungsi kompleks *loss* dan *activation* yang ingin diimplementasikan. Tentunya, secara praktis, untuk alasan *memory* dan efisiensi, terkadang lebih baik untuk langsung mendefinisikan fungsi (dan `Tensor` yang di-output-nya) secara manual. Berikut implementasi kelas ini secara lebih rinci.

Tabel 3: Kelas `Tensor`

Nama	**Tipe**	**Signature**	**Deskripsi**
`data`	atribut	`ndarray`	Nilai (seperti bobot) yang disimpan `Tensor`.
`grad`	atribut	`ndarray`	Gradien (jika berlaku) yang disimpan `Tensor`.
`_children`	atribut	`tuple`	Simpul anak dalam graf komputasi *autodiff*.
`_backward`	atribut	`callable`	Propagasi nilai gradien pada simpul induk dalam graf komputasi *autodiff*.
`__init__`	metode	`(data, _children=())`	Konstruktor kelas `Tensor`.
`__add__`	metode	`(other)`	Penjumlahan dua `Tensor`.
`__radd__`	metode	`(other)`	Penjumlahan terbalik dua `Tensor`.
`__neg__`	metode	`()`	Negatif dari `Tensor`.
`__sub__`	metode	`(other)`	Selisih dua `Tensor`.
`__mul__`	metode	`(other)`	Perkalian dua `Tensor`.
`__rmul__`	metode	`(other)`	Perkalian terbalik dua `Tensor`.
`__truediv__`	metode	`(other)`	Pembagian dua `Tensor`.
`__matmul__`	metode	`(other)`	Perkalian matriks dua `Tensor`.
`__rmatmul__`	metode	`(other)`	Perkalian matriks terbalik dua `Tensor`.
`__pow__`	metode	`(exponent)`	Perpangkatan `Tensor` dengan eksponen.
`sum`	metode	`(axis=None, keepdims=False)`	Penjumlahan elemen matriks `Tensor`.
`__abs__`	metode	`()`	Nilai mutlak `Tensor`.

Nama	Tipe	Signature	Deskripsi
`backward`	metode	`()`	Lakukan *backpropagation* seluruh graf komputasi dengan melakukan *topological sort* dan memanggil `_backward` individu secara berurutan.
`__repr__`	metode	`()`	Display `Tensor`.

Selanjutnya, kami membuat kelas `GradMode` dan `no_grad` untuk menjadi *state variable*, mirip dengan hook `useState` di `React`. Ini agar kami bisa mengendali apakah suatu *forward* atau *backward pass* akan dilakukan dengan juga meng-update nilai gradien juga atau tidak. Maka, di setiap pendefinisian operasi pada `Tensor`, kami melakukan cek apakah nilai gradien akan dikomunikasikan mundur pada graf komputasi *autodiff* atau tidak. Ini bertujuan untuk mengurangi *lag* dan *memory* saat hanya ingin melakukan validasi atau prediksi (tanpa *training*).

Tabel 4: Kelas `GradMode`

Nama	Tipe	Signature	Deskripsi
`enabled`	atribut	`bool`	Toggle mode perhitungan gradien.

Tabel 5: Kelas `no_grad`

Nama	Tipe	Signature	Deskripsi
`__enter__`	metode	`()`	Masuk mode menghitung gradien (untuk *train*).
`__exit__`	metode	`()`	Keluar mode menghitung gradien (untuk *predict*).

2.1.2. `activation.py`

Di file `activation.py`, kami implementasikan secara manual fungsi-fungsi aktivasi yang dapat digunakan pada FFNN. Seperti diskusi di atas, kita tahu bahwa cara paling mudah adalah untuk mendefinisikan suatu aktivasi σ menggunakan operasi-operasi dasar yang sudah ada di kelas `Tensor`. Tetapi, melakukannya secara manual (yang berarti harus dihitung turunan fungsi-fungsi aktivasi ini) membuat perhitungan FFNN lebih efisien dan ramah *memory*.

Tabel 6: Fungsi Global `activation.py`

Metode	Parameter	Return	Deskripsi
`linear`	`h` (`Tensor`)	`Tensor`	$\text{Linear}(h) = h$
`relu`	`h` (`Tensor`)	`Tensor`	$\text{ReLU}(h) = \max(0, h)$
`sigmoid`	`h` (`Tensor`)	`Tensor`	$\sigma(h) = \dfrac{1}{1 + e^{-h}}$
`tanh`	`h` (`Tensor`)	`Tensor`	$\tanh(h) = \dfrac{e^{h} - e^{-h}}{e^{h} + e^{-h}}$

Metode	Parameter	Return	Deskripsi
softmax	h (Tensor)	Tensor	$\text{Softmax}(h)_i = \dfrac{e^{h_i}}{\sum_j e^{h_j}}$
swish	h (Tensor)	Tensor	$\text{Swish}(h) = h \cdot \sigma(h)$ [6]
gelu	h (Tensor)	Tensor	$\text{GeLU}(h) = h \cdot \dfrac{1}{2}\left[1 + \text{erf}\left(\dfrac{h}{\sqrt{2}}\right)\right]$ [1]

Turunan fungsi-fungsi aktivasi di atas bisa dihitung dengan kalkulus secara mudah. Sedangkan turunan dari GeLU diberikan di [1].

2.1.3. initialize.py

File initialize.py memberikan metode-metode untuk menginisialisasikan bobot (atau bias) dari suatu layer. Berikut cara inisialisasi yang diimplementasikan. Rumus pada deksripsi berlaku *entry-wise* secara *independent and identically distributed* (iid).

Tabel 7: Fungsi Global initialize.py

Metode	Parameter	Return	Deskripsi
zero	(rows, cols, params=None, rng=None)	Tensor	$\texttt{zero}(r, c) = \mathbf{0}_{r \times c}$
uniform	(rows, cols, params=None, rng=None)	Tensor	$\texttt{uniform}(r, c) \sim \mathcal{U}(\text{low}, \text{high})$
normal	(rows, cols, params=None, rng=None)	Tensor	$\texttt{normal}(r, c) \sim \mathcal{N}(\mu, \sigma^2)$
xavier	(rows, cols, params=None, rng=None)	Tensor	$\texttt{xavier}(r, c) \sim \mathcal{U}\left(-\sqrt{\dfrac{6}{r+c}}, \sqrt{\dfrac{6}{r+c}}\right)$ [2]
he	(rows, cols, params=None, rng=None)	Tensor	$\texttt{he}(r, c) \sim \mathcal{N}\left(0, \dfrac{2}{r}\right)$ [3]

2.1.4. loss.py

File loss.py mengimplementasikan beberapa fungsi *loss*. Sekali lagi, kami membuatnya secara manual (daripada dengan operasi Tensor), dengan menghitung dengan tangan turunan-turunan dari fungsinya dengan kalkulus standar.

Tabel 8: Fungsi Global `loss.py`

Metode	Parameter	Return	Deskripsi
`mse`	`(pred, target)`	`Tensor`	$\texttt{mse}\,(y, \hat{y}) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
`bce`	`(pred, target)`	`Tensor`	$\texttt{bce}\,(y, \hat{y}) = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i))$
`cce`	`(pred, target)`	`Tensor`	$\texttt{cce}\,(y, \hat{y}) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C} y_{ij} \log \hat{y}_{ij}$

2.1.5. `optimizer.py`

Di file `optimizer.py`, dibangun metode optimisasi (*descent* pada fungsi *loss*) secara modular. Untuk melakukan itu, kita membuat kelas induk `Optimizer`. Lalu, metode khususnya menjadi kelas-kelas anak yang meng-*inherit* fitur dan metode dari kelas induk tersebut. Berikut rinciannya.

Tabel 9: Daftar Kelas pada `optimizer.py`

Kelas	Atribut	Deskripsi
`Optimizer`	`parameters`, `alpha`, `params`	Parent class untuk suatu metode optimisasi.
`SGD`	-	*Stochastic Gradient Descent* standar.
`Adam`	`beta1`, `beta2`, `epsilon`, `m`, `v`, `t`	Metode *Adaptive Moment Estimation* (Adam) sesuai paper ini.

Kelas umum `Optimizer` sendiri perlu memperhatikan *learning rate* dan hyperparameter khusus lainnya untuk melakukan optimisasi pada `parameter` yang diinginkan.

Tabel 10: Kelas `Optimizer`

Nama	Tipe	Signature	Deskripsi
`parameters`	atribut	`list`	Parameter (bobot, bias, etc.) yang ingin dioptimisasikan.
`alpha`	atribut	`float`	Learning rate.
`params`	atribut	`dict`	Hyperparameter metode seperti $\beta_1, \beta_2, \varepsilon$ di `adam`.
`__init__`	metode	`(parameters, learning_rate=0.01, **kwargs)`	Konstruktor `Optimizer`.
`step`	metode	`()`	Satu langkah descent (satu iterasi).

Nama	Tipe	Signature	Deskripsi
`zero_grad`	metode	`()`	Mengenolkan nilai parameter.

Berikut implementasi dari dua metode yang dibuat: *Stochastic Gradient Descent* dan *Adaptive Moment Estimation* (Adam).

Tabel 11: Kelas `SGD`

Nama	Tipe	Signature	Deskripsi
`step`	metode	`()`	$\theta_t := \theta_{t-1} + \alpha \nabla_\theta \mathcal{L}(\theta_{t-1})$

Tabel 12: Kelas `Adam`

Nama	Tipe	Signature	Deskripsi
`beta1`	atribut	`float`	Koefisien peluruhan untuk taksiran momen pertama.
`beta2`	atribut	`float`	Koefisien peluruhan untuk taksiran momen kedua.
`epsilon`	atribut	`float`	Koreksi numerik (agar tidak membagi dengan nol).
`m`	atribut	`list`	Array taksiran momen pertama bias.
`v`	atribut	`list`	Array taksiran momen kedua bias.
`t`	atribut	`int`	Iterasi descent.
`__init__`	metode	`(parameters, learning_rate=0.01, **kwargs)`	Konsruktor `Adam`.
`step`	metode	`()`	$g_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1})$ $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t$ $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$ $\hat{m}_t \leftarrow m_t / (1-\beta_1^t)$ $\hat{v}_t \leftarrow v_t / (1-\beta_2^t)$ $\theta_t \leftarrow \theta_{t-1} + \alpha \cdot \hat{m}_t \,/\, \left(\sqrt{\hat{v}_t} + \varepsilon\right)$

2.1.6. `nn.py`

File `nn.py` menggabungkan semua komponen-komponen dari FFNN menjadi satu module. Secara khusus, dibuat kelas induk `Module`, yang akan di-*inherit* oleh `Layer`, `RMSNorm`, dan `MLP`. Inilah fungsi FFNN yang diinginkan.

Tabel 13: Daftar Kelas pada `nn.py`

Kelas	Atribut	Deskripsi
`Module`	-	Parent class untuk suatu komponen dalam arsitektur FFNN.

Kelas	Atribut	Deskripsi
Layer	n_neurons, input_size, weight_init, bias_init, seed, l1, l2, weights, bias, activation	Layer (*hidden* atau *output*) pada FFNN, yang dilengkapi dengan bobot dan bias.
RMSNorm	eps, input_size, n_neurons, gamma	Lapisan yang melakukan normlisasi RMSNorm pada FFNN.
MLP	seed, input_size, layers, loss_fn, optimizer	Seluruh model FFNN, yang sudah dilengkapi dengan arsitekturnya, serta cara untuk melakukan *training* dan *prediction*.

Kelas `Module` sendiri perlu bisa menyimpan semua parameter (bobot dan bias misalnya) yang dimilikinya, sesuai spesifikasi tugas.

Tabel 14: Kelas `Module`

Nama	Tipe	Signature	Deskripsi
parameters	metode	()	Daftar semua parameter (bobot, bias, etc.) dari suatu modul.
zero_grad	metode	()	Mengenolkan nilai parameter.

Kelas `Layer` adalah satu lapisan L_i pada FFNN. Diperlukan banyak atribut untuk merincikan arsitektur yang diiginkan.

- Layer perlu bisa menyimpan jumlah neuronnya (misalkan m) dan besar inputnya (misalkan n). Ini berarti layer tersebut adalah suatu fungsi $\mathbb{R}^n \to \mathbb{R}^m$.
- Layer bisa menyimpan bobot ($\mathbb{R}^{m \times n}$) dan bias ($\mathbb{R}^n$) dalam bentuk `Tensor`. Inisialisasi mereka dapat dispesifikasikan. Secara *default*, inisialisasi bobot dengan metode Xavier (seperti pada `Keras`), dan inisialisasi bias dengan nol (seumumnya). Alasannya adalah bahwa metode Xavier untuk bobot bagus untuk mempertahankan variansi output dan input suatu layer [2], sedangkan inisialisi nol untuk bias memang asumsi dari metode Xavier pada papernya.
- Regularisasi bisa dipilih *per layer* saja (bukan per neuron) dengan atribut `l1` dan `l2`. Alasannya karena pengaruh bobot pada MLP hanya bermakna jika dilakukan secara kolektif dalam satu layer setidaknya, bukan secara individu. Selebihnya, ini juga standar di package-package *machine learning* seumumnya.
- Layer punya fungsi aktivasi sendiri yang langsung disimpan dalam atribut sebagai fungsi.

Detailnya pada tabel berikut.

Tabel 15: Kelas `Layer`

Nama	Tipe	Signature	Deskripsi
`n_neurons`	atribut	`int`	Jumlah nueron pada layer.
`input_size`	atribut	`int` | `None`	Jumlah neuron pada layer sebelumnya.
`weight_init`	atribut	`str` | `dict`	Metode inisialisasi bobot.
`bias_init`	atribut	`str` | `dict`	Metode inisialisasi bias.
`seed`	atribut	`int` | `None`	Seed untuk randomization.
`l1`	atribut	`float`	Bobot regularisasi L1 (λ).
`l2`	atribut	`float`	Bobot regularisasi L2 (λ).
`weights`	atribut	`Tensor` | `None`	Matriks bobot pada layer.
`bias`	atribut	`Tensor`	Vektor bias pada layer.
`activation`	atribut	`callable`	
`__init__`	metode	`(n_neurons, input_size=None, activation='relu', weight_init='xavier', bias_init='zero', seed=None, l1=0.0, l2=0.0)`	Konsruktor `Layer`.
`_initialize_parameters`	metode	`(rows, cols, init_method, rng)`	Fungsi yang melakukan inisialisasi bobot dan bias.
`parameters`	metode	`()`	Daftar semua bobot dan bias dari suatu layer.
`regularization_loss`	metode	`()`	Menghitung kontribusi bobot layer pada fungsi *loss*. Dicatat bahwa perhitungan dilakukan dengan operasi-operasi pada `Tensor`, bukan secara manual.

Nama	Tipe	Signature	Deskripsi
`__call__`	metode	`(input)`	Melakukan *forward pass* pada layer.

Ada juga lapisan `RMSNorm` yang dibuat secara terpisah dari `Layer`. Alasannya adalah kemudahan (daripada perlu menambahkan fitur pada `Layer` yang sudah jadi). Lalu, perhitungan yang dilakukan juga menggunakan operasi-operasi dasar `Tensor` daripada secara manual seperti untuk aktivasi dan *loss*. Lagi, alasannya adalah kemudahan (walaupun secara praktis ini berari graf kompuasi *autodiff* yang dibuat akan lebih dalam, sehingga mengambil *memory*).

Tabel 16: Kelas `RMSNorm`

Nama	Tipe	Signature	Deskripsi
`eps`	atribut	`float`	Galat numerik (agar tidak membagi dengan nol).
`input_size`	atribut	`int`	Jumlah neuron pada layer sebelumnya.
`n_neurons`	atribut	`int`	Jumlah neuron pada lapisan `RMSNorm`.
`gamma`	atribut	`Tensor`	Pengali skala statistik RMSNorm.
`__init__`	metode	`(size, eps=1e-8)`	Konstruktor `RMSNorm`.
`parameters`	metode	`()`	Daftar semua γ pada lapisan `RMSNorm`.
`regularization_loss`	metode	`()`	Return 0, hanya untuk alasan keberlangsungan kode.
`__call__`	metode	`(x)`	Melakukan *forward pass* pada lapisan `RMSNorm`: $\bar{a}_i = \frac{a_i}{\mathrm{RMS}(\mathbf{a})} g_i,$ $\mathrm{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n}\sum_{i=1}^{n} a_i^2}$ [7]

Terakhir, kelas `MLP` menggabungkan layer-layer (dan `RMSNorm` jika ada). Secara matematis, dia hanyalah komposisi layer-layer: $L_\ell \circ L_{\ell-1} \circ \cdots \circ L_1$. Berikut beberapa hal untuk dicatat tentang implementasinya:

- Pembuatan suatu `MLP` cukup dengan memberikan layer-layer yang diinginkan, dengan kemungkinan hyperparameter juga. Penambahan layer setelah konstruktor juga bisa dengan metode `add`. Ini bisa diimplementasikan karena ada fungsi bantuan `_add_input` dan `_cascade_input`, sehingga saat pembuatan FFNN, atribut `input_size` terdefinisi.
- Pemilihan fungsi *loss* dan metode optimisasi adalah dengan metode `compile`, mirip dengan package `Keras`.

- Kami bisa melakukan *training* dengan metode `fit` seperti di `scikit`. Lalu, prediksi juga dengan memanggil kelasnya sendiri `MLP(data)` seperti suatu fungsi.
- Semua bobot (dari tiap layer) bisa dilihat di `parameters`. Tetapi, lebih baik melakukan visualisasi (nilai maupun gradien) dengan fungsi `plot_weights` dan `plot_gradients`. Bobot tersebut bahkan bisa disimpan dalam file dengan `save`, dan dimuat dari suatu file dengan `load`.

Detailnya pada tabel berikut.

Tabel 17: Kelas `MLP`

Nama	**Tipe**	**Signature**	**Deskripsi**
`seed`	atribut	`int` | `None`	Seed untuk randomization.
`input_size`	atribut	`int` | `None`	Dimensi layer *input*.
`layers`	atribut	`list`	Array berisi semua layer (*hidden* dan *output*).
`loss_fn`	atribut	`callable` | `None`	Fungsi *loss* yang digunakan.
`optimizer`	atribut	`Optimizer` | `None`	Metode optimisasi yang digunakan.
`__init__`	metode	`(layers=None, input_size=None, seed=None)`	Konstruktor `MLP`.
`parameters`	metode	`()`	Daftar semua parameter (bobot, bias, dan γ).
`add`	metode	`(layers)`	Menambahkan layer pada `MLP`.
`_add_input`	metode	`(layer, input_size)`	Update `input_size` agar sesuai.
`_push_input`	metode	`(prev_layer, layer)`	Update `input_size` berdasarkan `n_neurons` layer sebelumnya.

Nama	Tipe	Signature	Deskripsi
_cascade_inputs	metode	(layers, initial_input_size)	Melakukan _push_input pada seluruh MLP.
compile	metode	(optimizer, loss)	Inisialisasi pemilihan fungsi *loss* dan metode optimisasi.
_regularization_loss	metode	()	Menghitung pengaruh regularisasi setiap layer pada *loss*.
fit	metode	(X, y, batch_size=32, epochs=100, validation_data=None, verbose=1)	Melakukan *training* (dan validasi) terhadap data.
plot_weights	metode	(layer_indices=None)	Memplot nilai bobot layer-layer pada MLP.
plot_gradients	metode	(layer_indices=None)	Memplot gradien bobot layer-layer pada MLP.
save	metode	(filepath)	Menyimpan parameter MLP dalam suatu file.
load	metode	(filepath)	Memuat parameter MLP dari suatu file.
summary	metode	()	Memprint arsitektur model MLP.
__call__	metode	(input)	Melakukan *forward pass* seluruh FFNN.

2.2. Hasil Pengujian

2.2.1. Pengaruh *Depth* dan *Width*

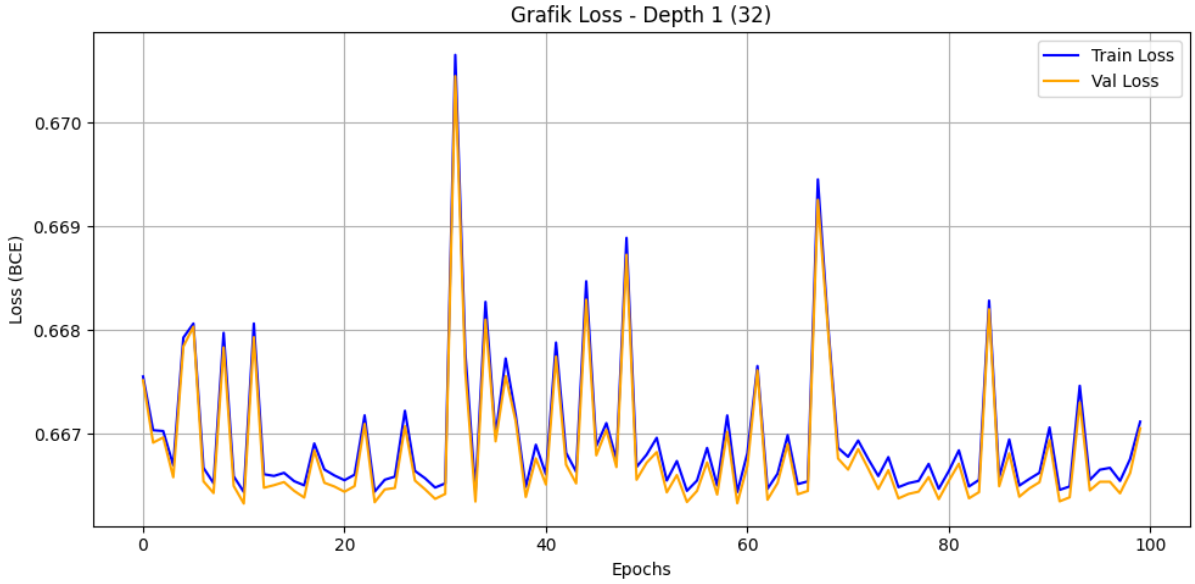
Akan ditentukan pengaruh *hyperparameter* pertama, yakni banyak layer/*depth* dan banyak neuron per layer/*width*. Dipilih tiga variasi kombinasi *width* dengan *depth* tetap

dan tiga variasi kombinasi *depth* dengan *width* tetap. Nilai *learning rate* yang digunakan sebesar 0.01. Variasi yang dipilih diantaranya sebagai berikut.

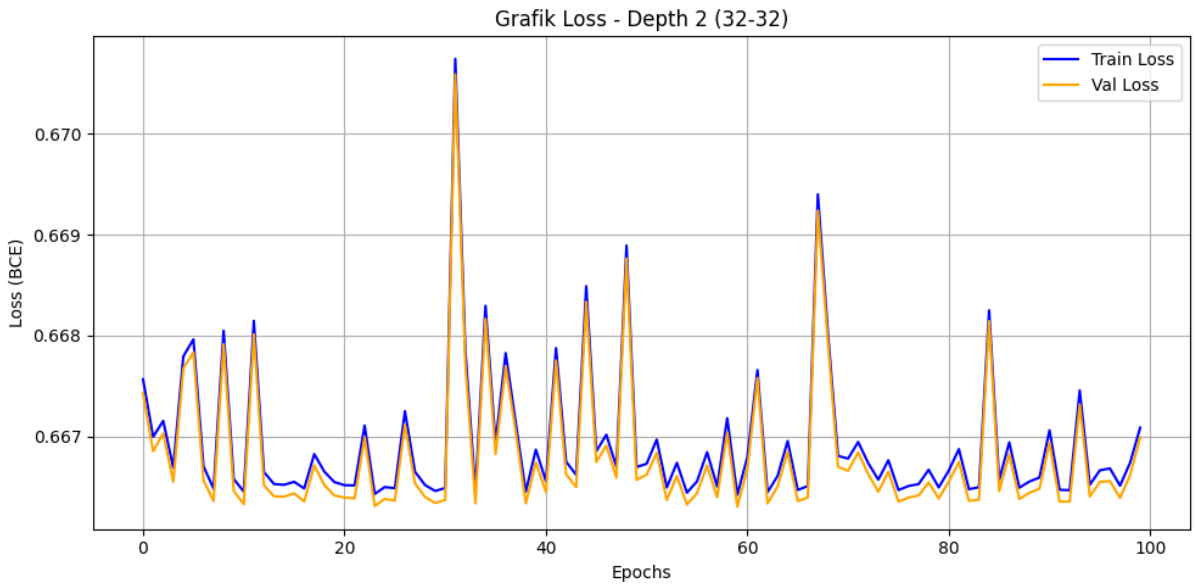
Tabel 18: Variasi untuk Pengujian *Depth* dan *Width*

Variasi	***Depth***	***Width***
1	1 *hidden layer*	32 neuron
2	2 *hidden layer*	32 neuron
3	3 *hidden layer*	32 neuron
4	2 *hidden layer*	16 neuron
5	2 *hidden layer*	32 neuron
6	2 *hidden layer*	64 neuron

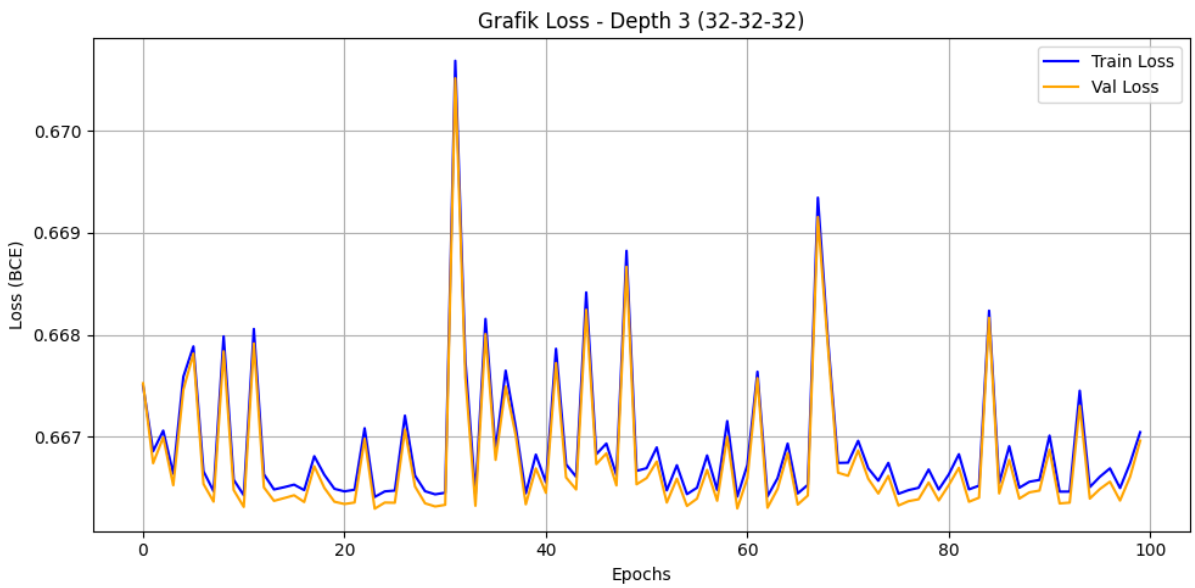
Hasil pengujian ditunjukkan pada grafik di bawah ini.-



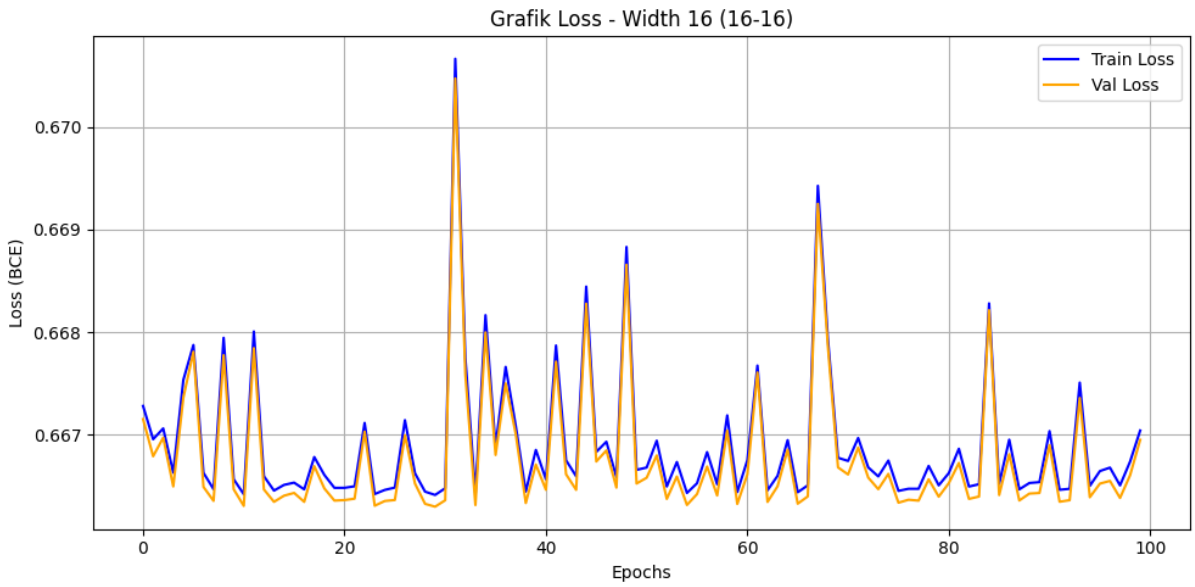
Gambar 2: Grafik *loss* untuk *depth* 1



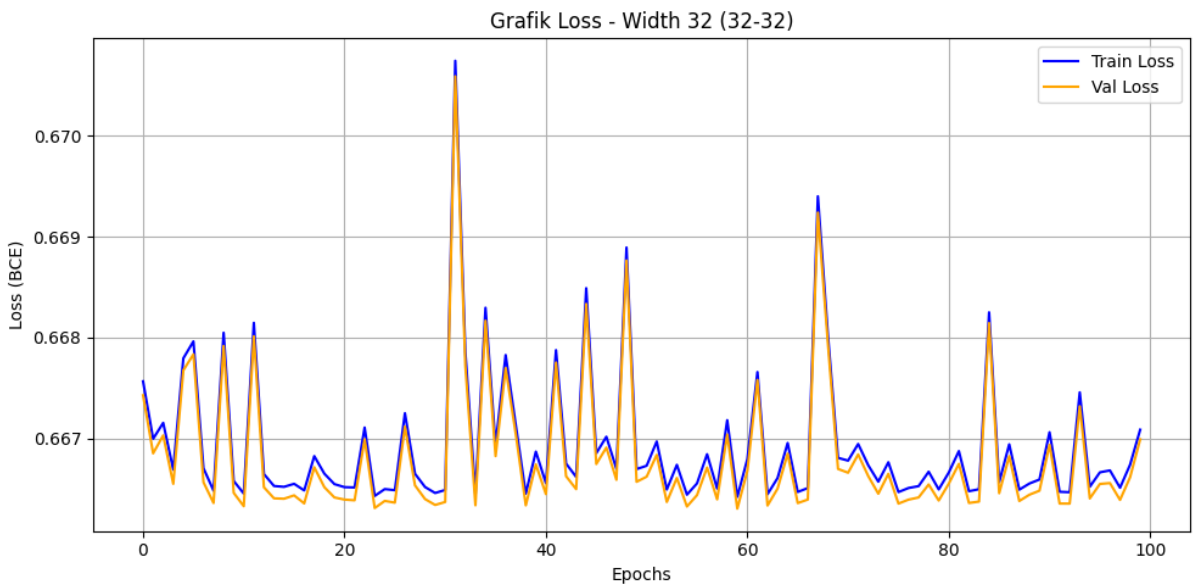
Gambar 3: Grafik *loss* untuk *depth* 2



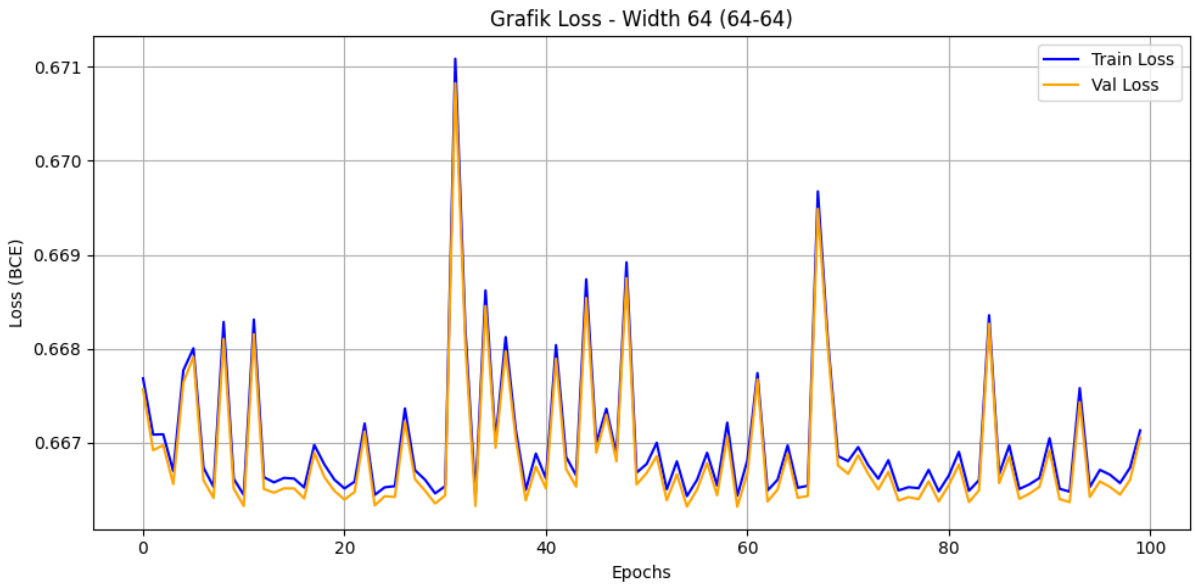
Gambar 4: Grafik *loss* untuk *depth* 3



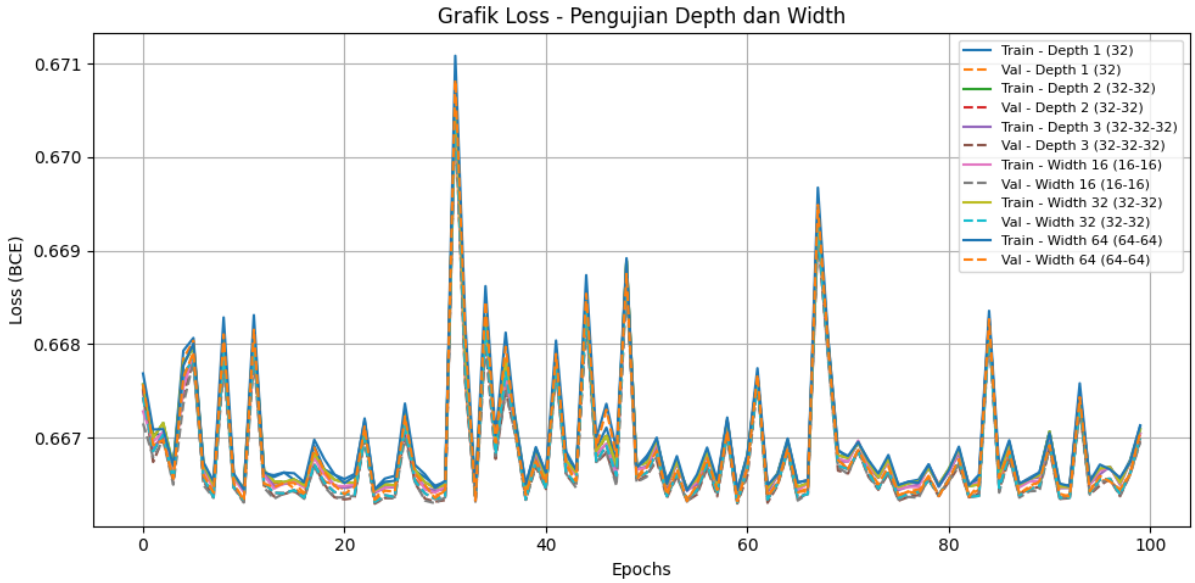
Gambar 5: Grafik *loss* untuk *width* 16



Gambar 6: Grafik *loss* untuk *width* 32



Gambar 7: Grafik *loss* untuk *width* 64



Gambar 8: Grafik *loss* gabungan variasi *depth* dan *width*

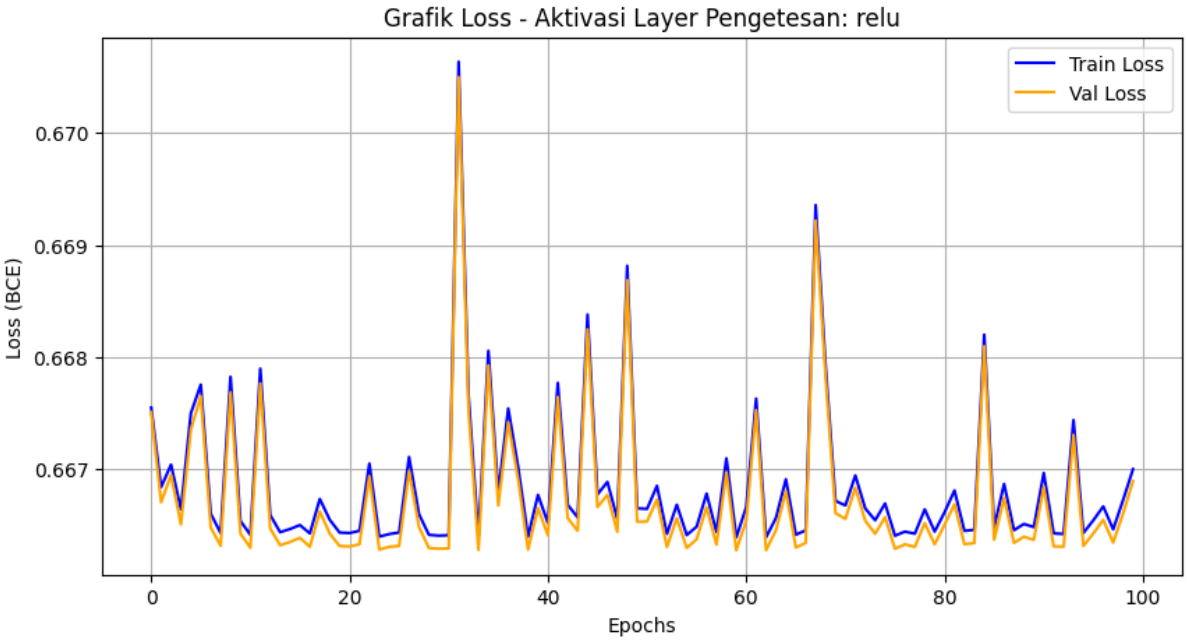
Berdasarkan grafik *training* dan *validation loss*, diperoleh bahwa grafik *loss* tidak menunjukkan kurva penurunan yang mulus. Terdapat lonjakan *loss* yang sangat tajam dan berulang di epoch yang persis sama untuk semua arsitektur, contohnya di sekitar epoch $10, 30, 45, 65$, dan 85. Puncak tertinggi nilai *loss* selalu terjadi di sekitar epoch 30. Garis *train loss* (biru) dan *validation loss* (oranye) bergerak sangat berdekatan dari awal hingga akhir epoch. Di beberapa titik, *validation loss* sedikit lebih rendah daripada *train loss*. Hal ini menunjukkan bahwa model tidak menghafal data latih sehingga tidak terjadi *overfitting*. Akan tetapi, model juga kesulitan untuk belajar lebih jauh atau dengan kata lain, terjadi *underfitting* dan nilai *loss* terjebak di titik minimum lokal. Rentang nilai *loss binary-cross entropy* sangat sempit, hanya berada pada $[0.666, 0.671]$.

Selanjutnya, perbandingan untuk hasil akhir prediksi di sini kita lihat dari nilai *loss binary-cross entropy* pada akhir pelatihan atau epoch 100. Dapat dilihat bahwa penambahan jumlah *layer* dari 1 menjadi 3 tidak memberikan perubahan yang signifikan

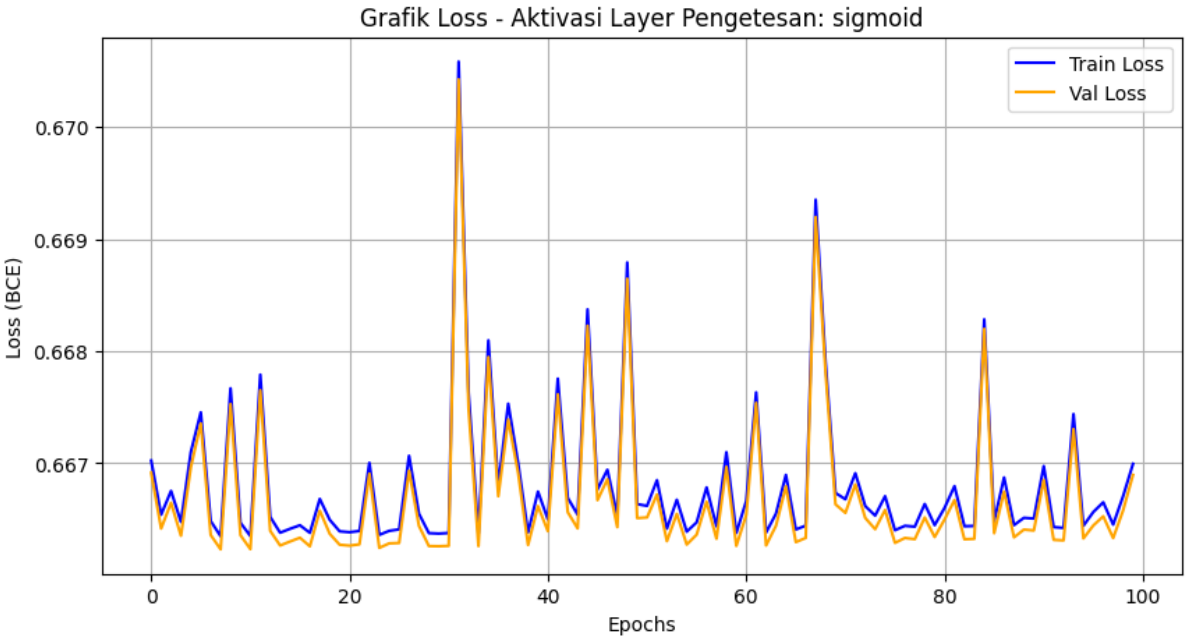
pada nilai akhir *loss*. Ketiga model berakhir pada kisaran nilai *loss* ~ 0.667. Selain itu, penambahan neuron pada *layer* dari 16 hingga 64 neuron juga memberikan hasil yang tidak berbeda secara signifikan. Nilai akhir *loss* tetap berada di kisaran ~ 0.667. Pada model dengan jumlah neuron sebanyak 64 dalam layer memberikan sedikit variasi tinggi pada lonjakan *loss* di epoch 30, yaitu mencapai 0.671, tetapi tetap tidak memperbaiki hasil akhir di epoch 100. Berdasarkan analisis di atas, kita menyimpulkan bahwa mengubah *depth* maupun *width* pada model ini tidak memberikan pengaruh yang signifikan terhadap performa model FFNN.

2.2.2. Pengaruh Fungsi Aktivasi

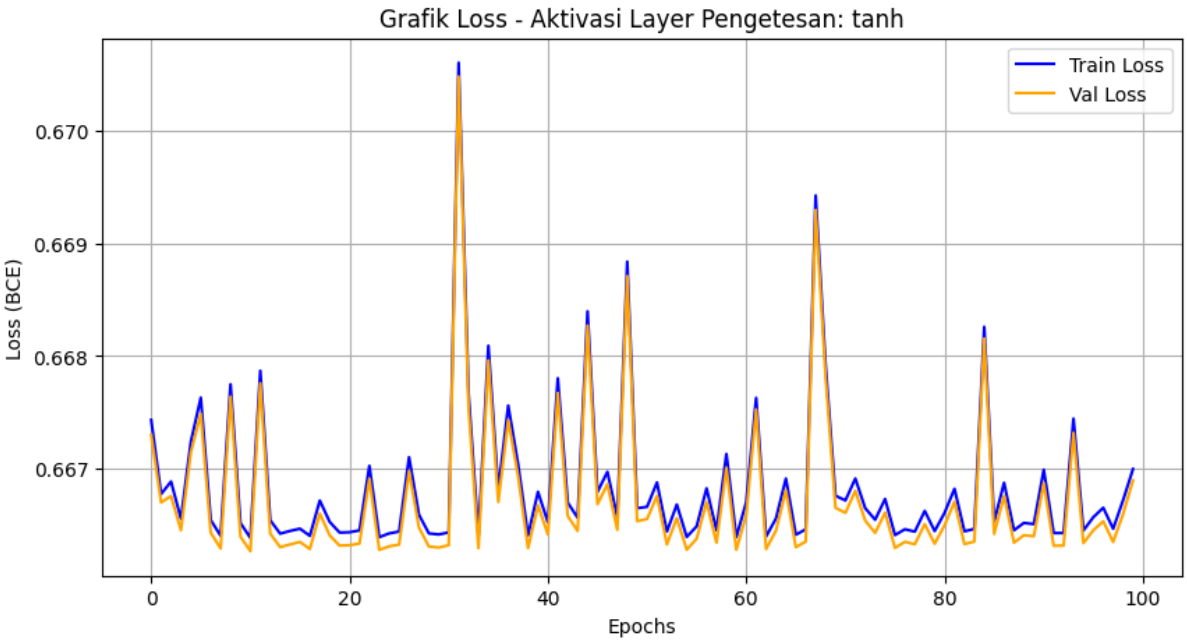
Akan ditentukan pengaruh dari penggunaan fungsi aktivasi pada *hidden layer*. Dipilih tiga *hidden layer* dengan fungsi aktivasi pada layer pertama dan ketiga dibuat tetap menggunakan fungsi ReLU, serta layer kedua sebagai layer pengujian. Pengujian dilakukan empat kali dengan layer pengujian diterapkan fungsi aktivasi ReLU, sigmoid, tanh, dan GeLU. Digunakan 3 *hidden layer* dengan 32 neuron dan *learning rate* sebesar 0.01 yang sama untuk keempat pengujian. Hasil pengujian ditunjukkan pada grafik di bawah ini.



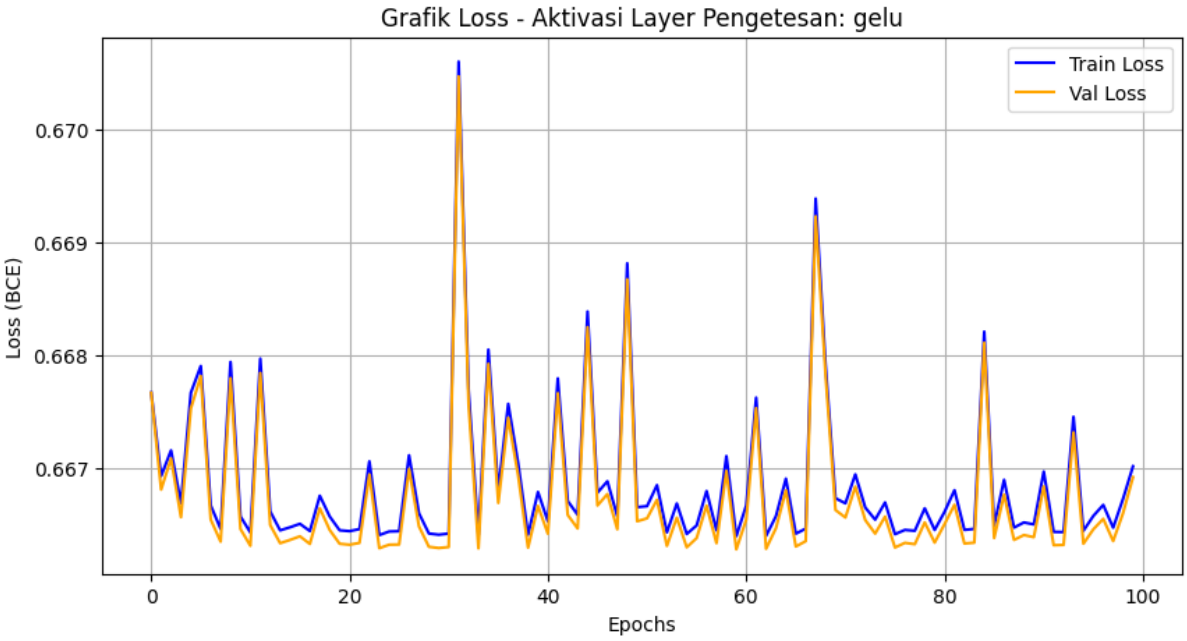
Gambar 9: Grafik *loss* untuk fungsi aktivasi ReLU pada *hidden layer*



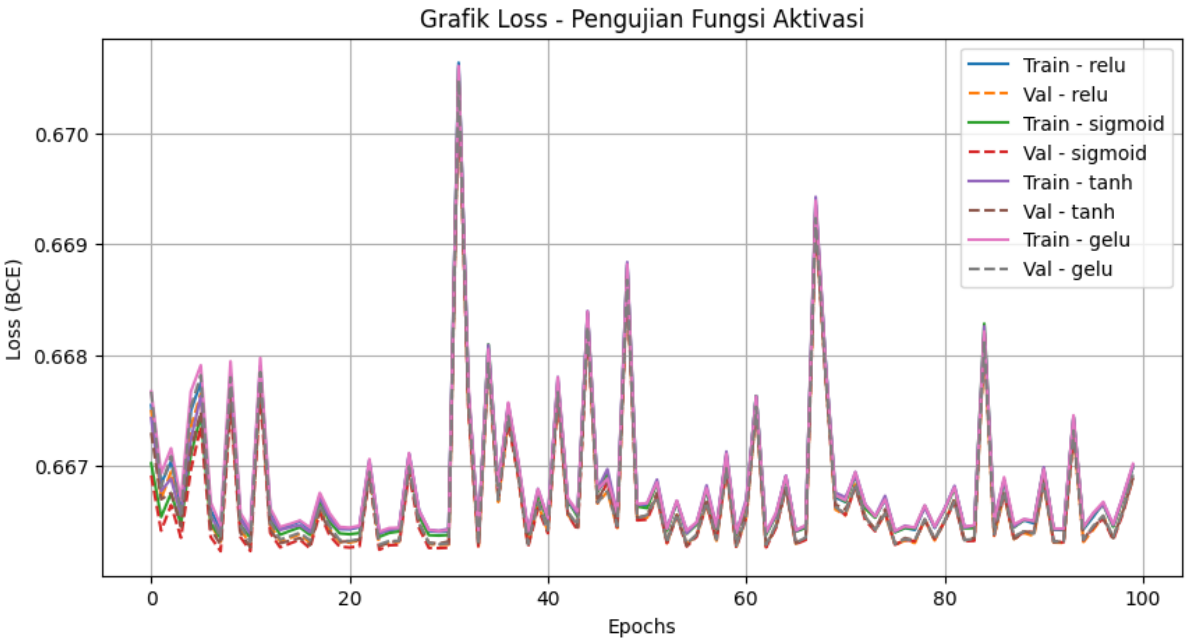
Gambar 10: Grafik *loss* untuk fungsi aktivasi sigmoid pada *hidden layer*



Gambar 11: Grafik *loss* untuk fungsi aktivasi tanh pada *hidden layer*

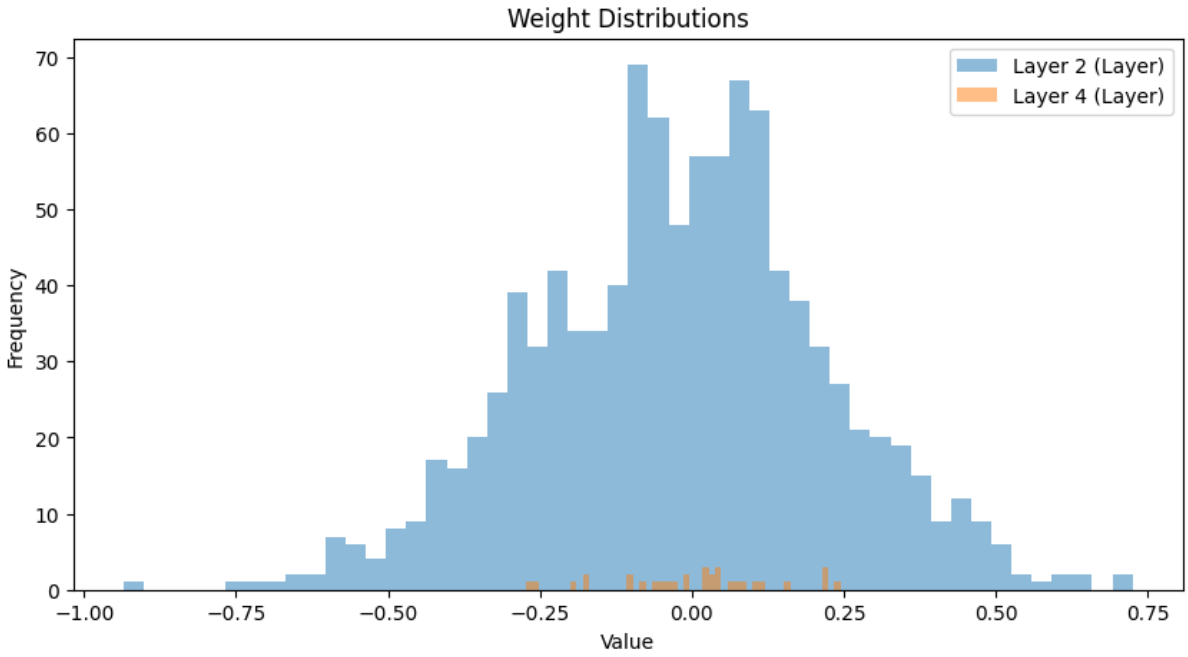


Gambar 12: Grafik *loss* untuk fungsi aktivasi GeLU pada *hidden layer*

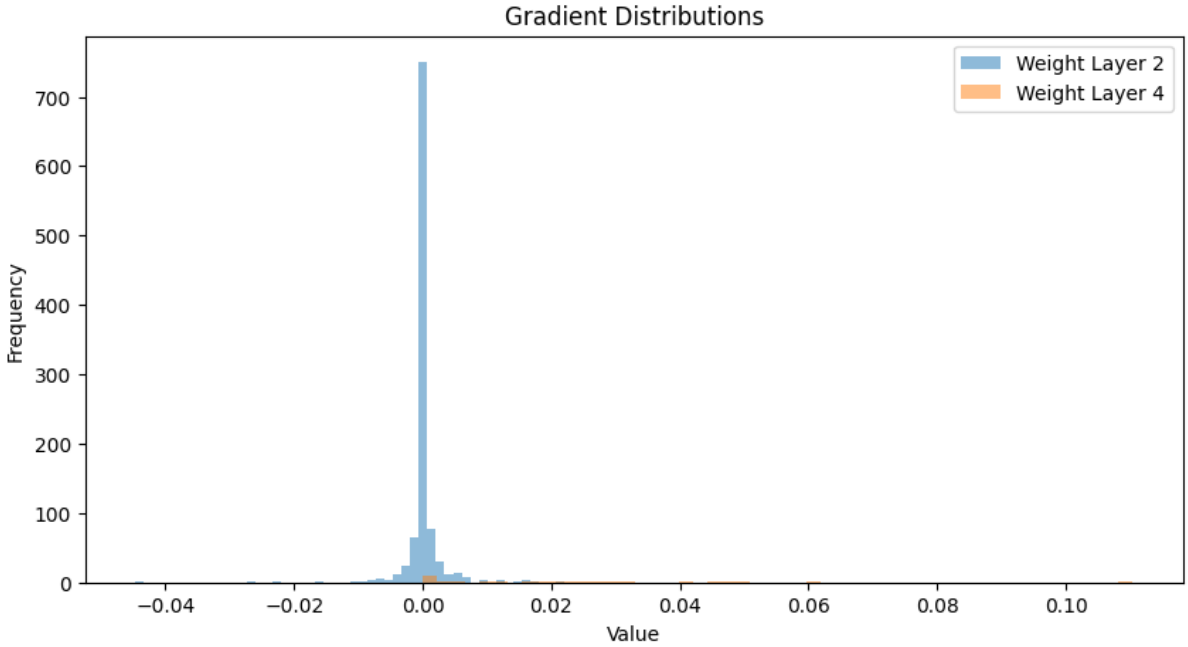


Gambar 13: Grafik *loss* gabungan keempat variasi fungsi aktivasi pada *hidden layer*

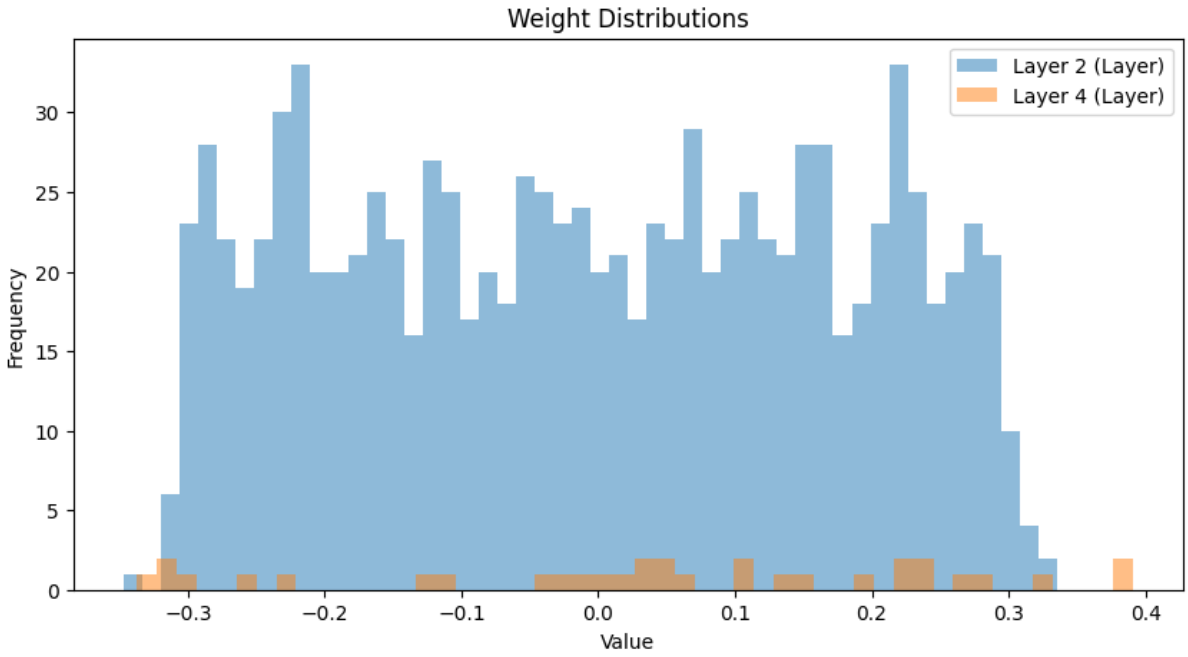
Berdasarkan grafik *training* dan *validation loss*, diperoleh bahwa perubahan fungsi aktivasi menjadi relu, sigmoid, tanh, maupun gelu tidak mengubah pola *loss* sama sekali. Lonjakan tajam tetap terjadi secara konsisten di epoch yang persis sama, terutama lonjakan tinggi terjadi di sekitar epoch 30. Selain itu, nilai *loss* pada akhir pelatihan untuk semua fungsi aktivasi hampir sama, yaitu pada kisaran angka ~ 0.667.



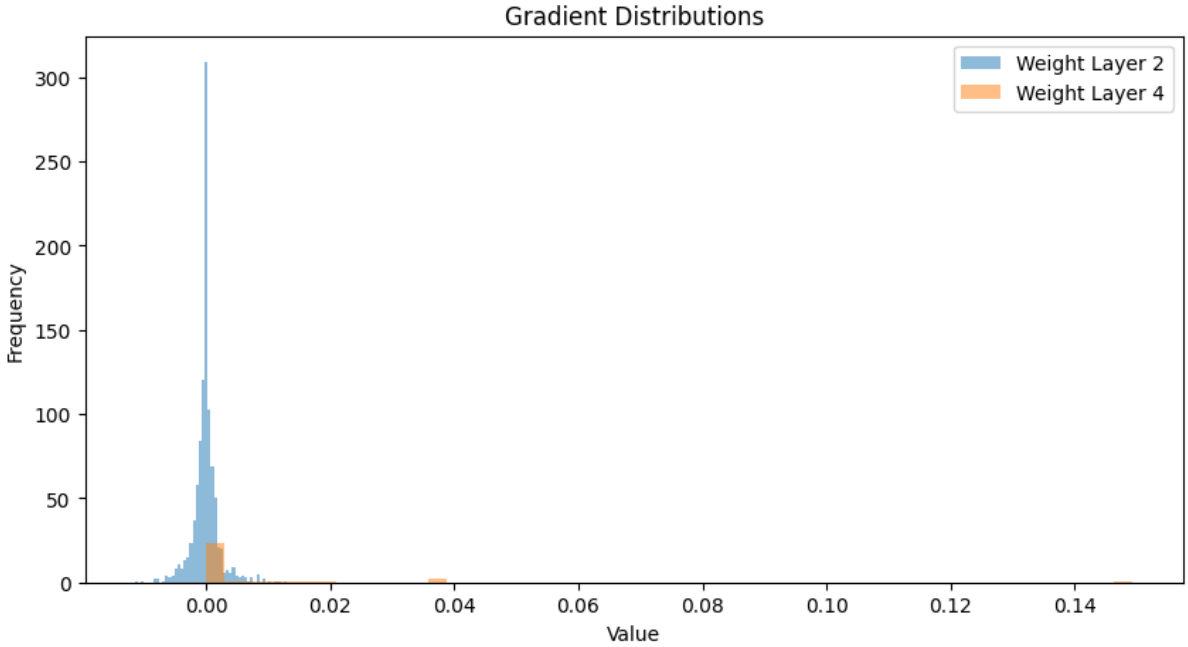
Gambar 14: Grafik distribusi bobot untuk fungsi aktivasi ReLU pada *hidden layer*



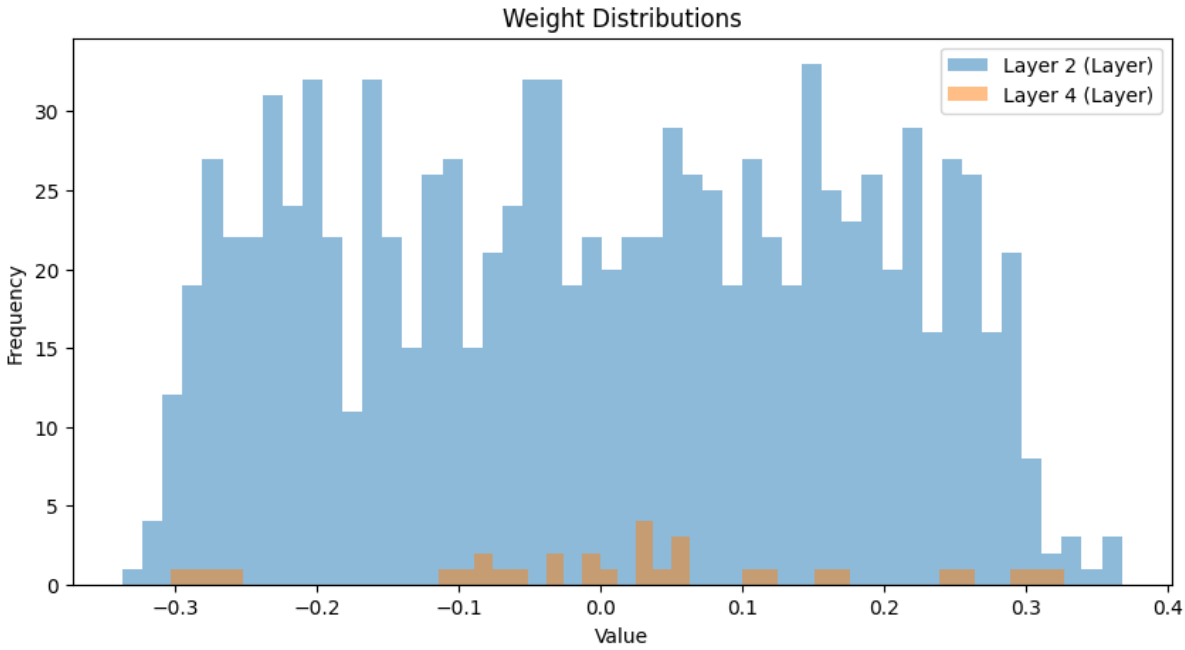
Gambar 15: Grafik distribusi gradient untuk fungsi aktivasi ReLU pada *hidden layer*



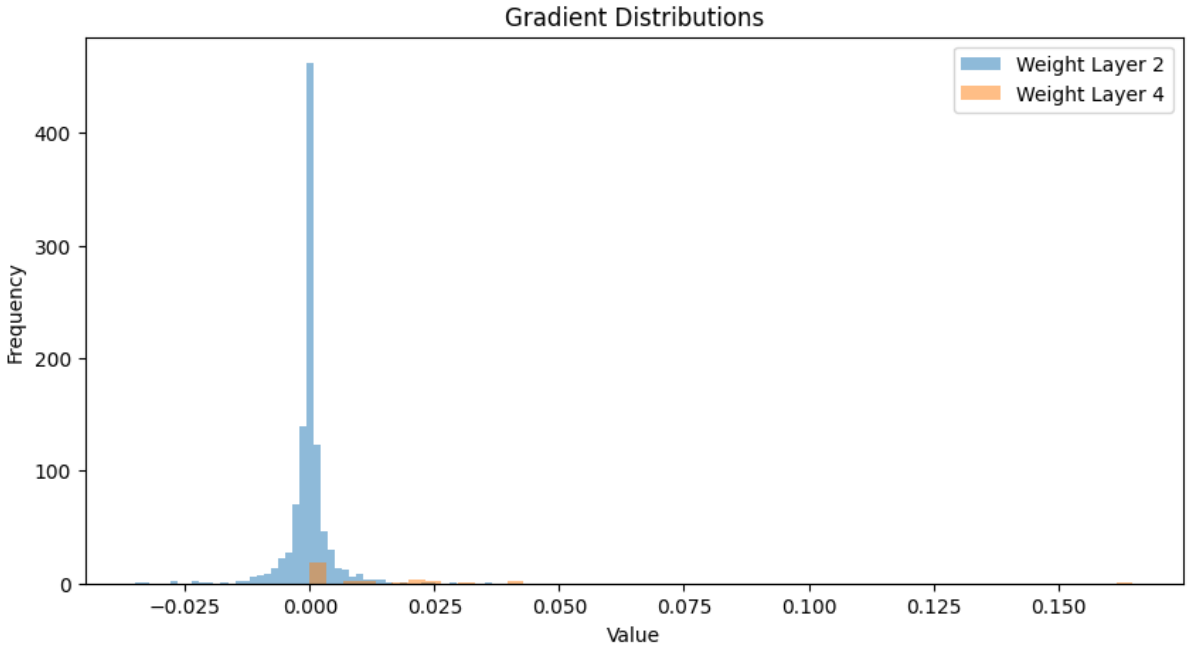
Gambar 16: Grafik distribusi bobot untuk fungsi aktivasi sigmoid pada *hidden layer*



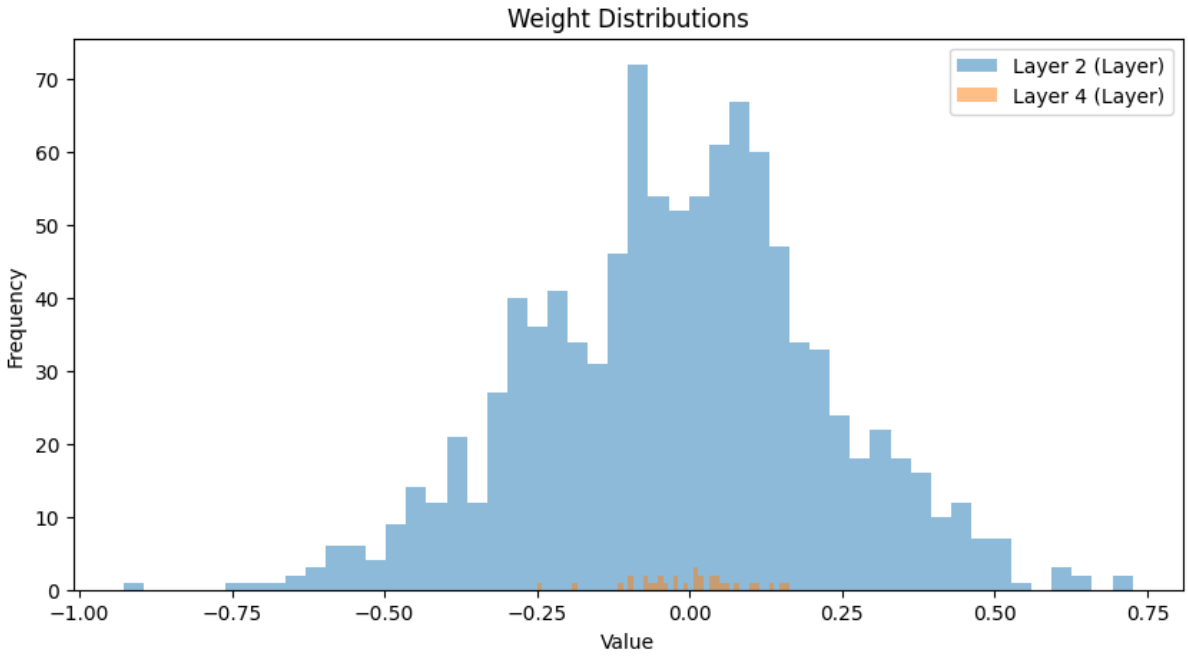
Gambar 17: Grafik distribusi gradient untuk fungsi aktivasi sigmoid pada *hidden layer*



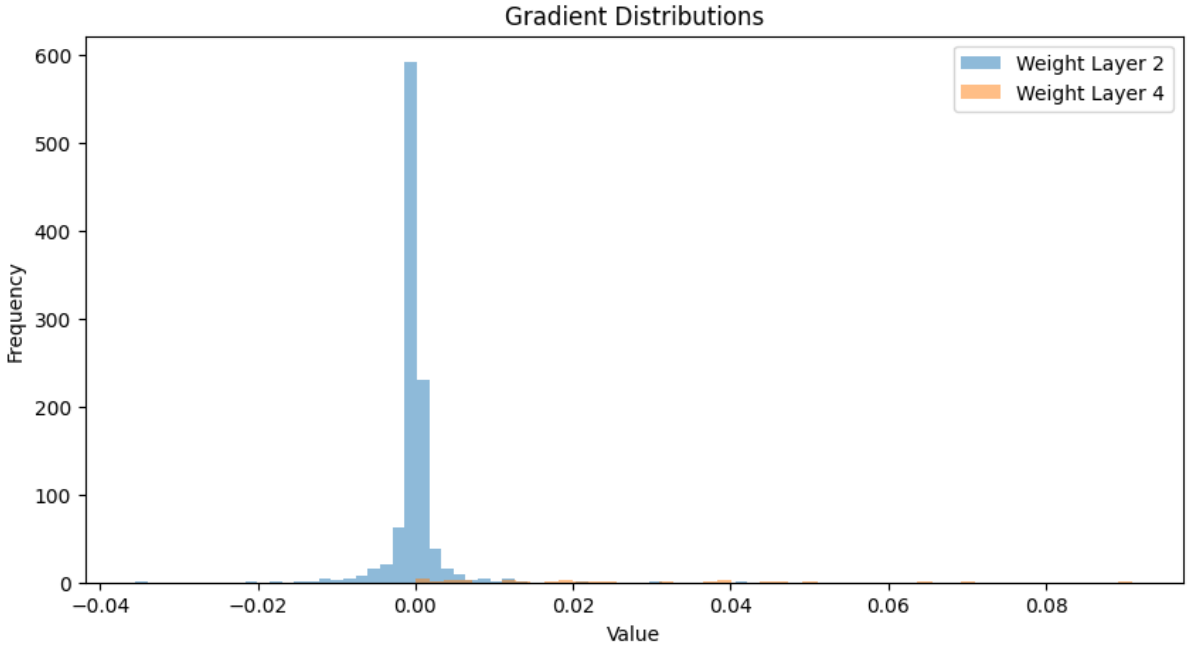
Gambar 18: Grafik distribusi bobot untuk fungsi aktivasi tanh pada *hidden layer*



Gambar 19: Grafik distribusi gradient untuk fungsi aktivasi tanh pada *hidden layer*



Gambar 20: Grafik distribusi bobot untuk fungsi aktivasi GeLU pada *hidden layer*

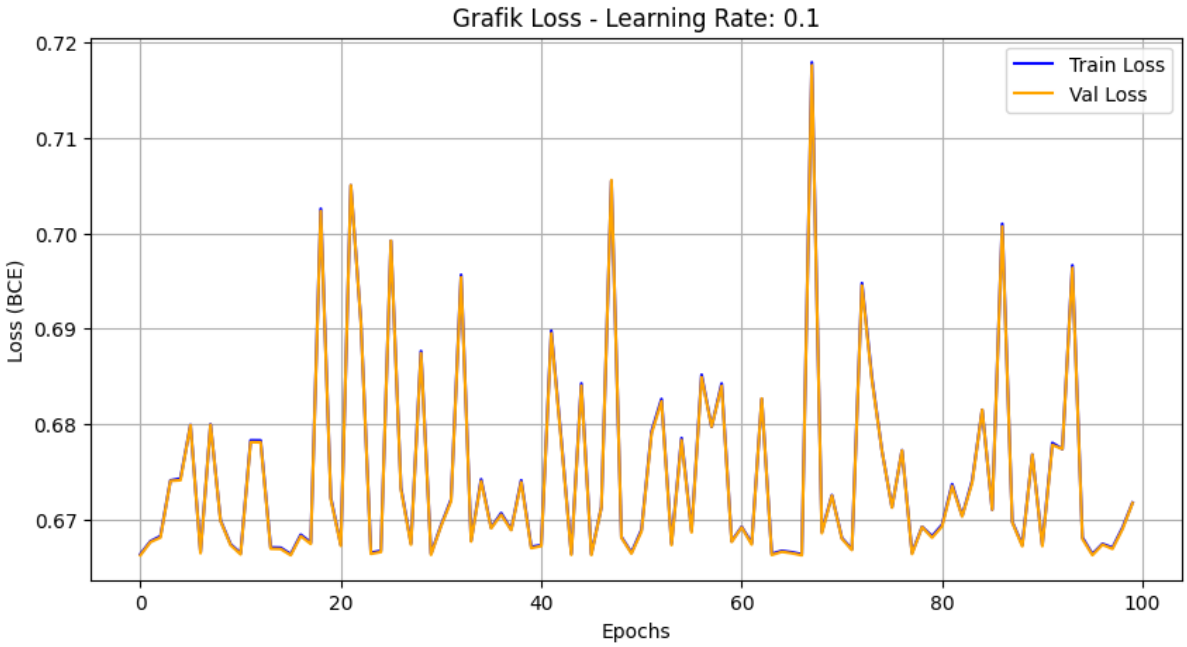


Gambar 21: Grafik distribusi gradient untuk fungsi aktivasi GeLU pada *hidden layer*

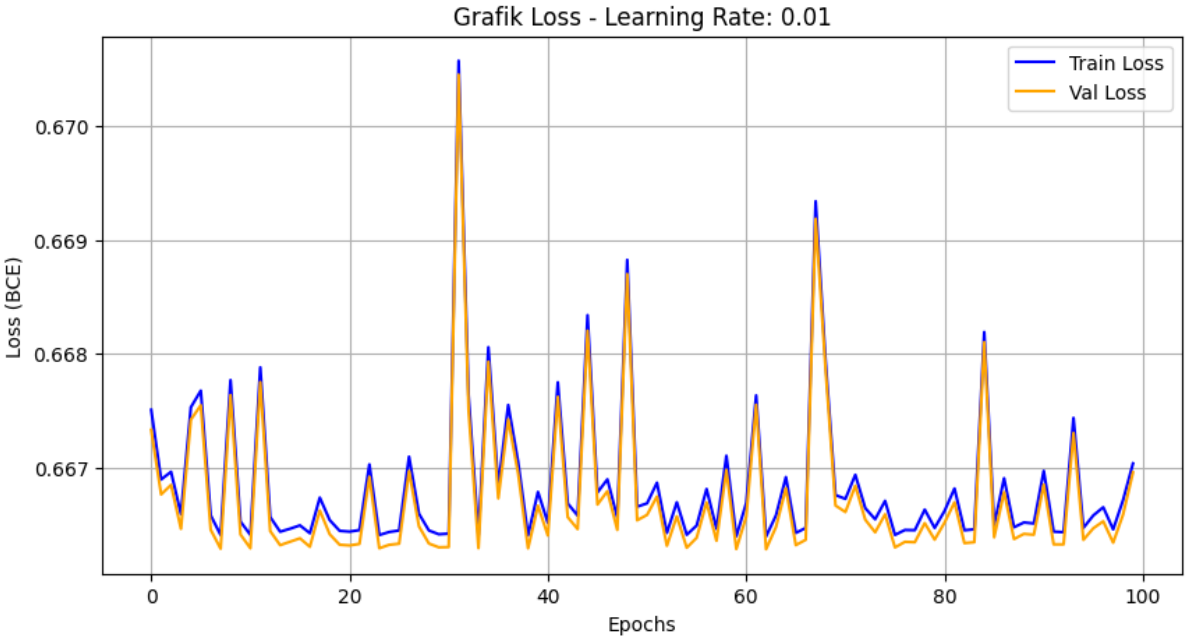
Selanjutnya, akan dilakukan analisis berdasarkan grafik distribusi bobot dan gradien. Pada *hidden layer* pertama, bobot tersebar cukup normal menyerupai kurva lonceng dari sekitar -0.75 hingga 0.75. Hal ini merupakan distribusi bobot inisialisasi yang wajar, seperti He atau Xavier *initialization*. Pada *output layer*, frekuensinya sangat kecil dan terpusat di sekitar angka 0. Pada *hidden layer* pertama, terdapat lonjakan frekuensi yang ekstrem tepat di angka 0. Artinya, nilai gradien yang mengalir kembali ke layer 2 mayoritas mendekati nol. Hal ini merupakan indikasi terjadinya *vanishing gradient* atau indikasi bahwa model sudah berhenti belajar. Jika gradiennya 0, bobot tidak akan di-*update* secara berarti oleh *optimizer*. Oleh karena itu, nilai *loss* tidak bisa turun dari angka 0.667 atau dengan kata lain, model terjebak di titik minimum lokal.

2.2.3. Pengaruh *Learning Rate*

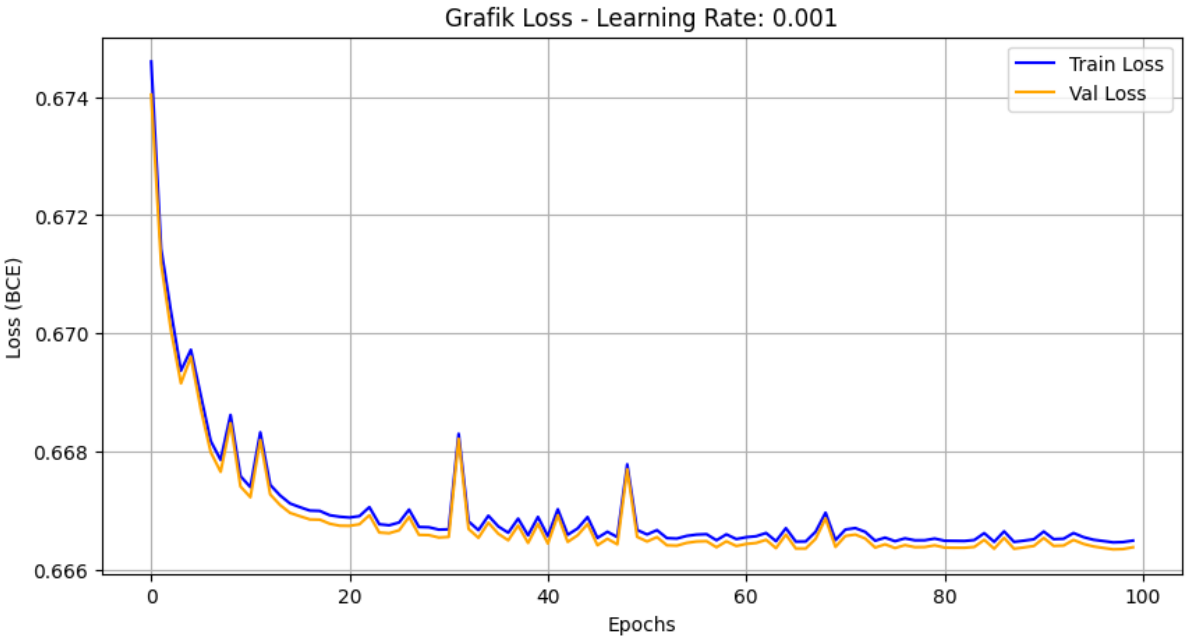
Akan ditentukan pengaruh dari penggunaan *learning rate* yang berbeda-beda. Dipilih tiga nilai variasi *learning rate*, yaitu $0.1, 0.01$, dan 0.001. Digunakan 2 *hidden layer* dengan 32 neuron yang sama untuk ketiga pengujian. Hasil pengujian ditunjukkan pada grafik di bawah ini.



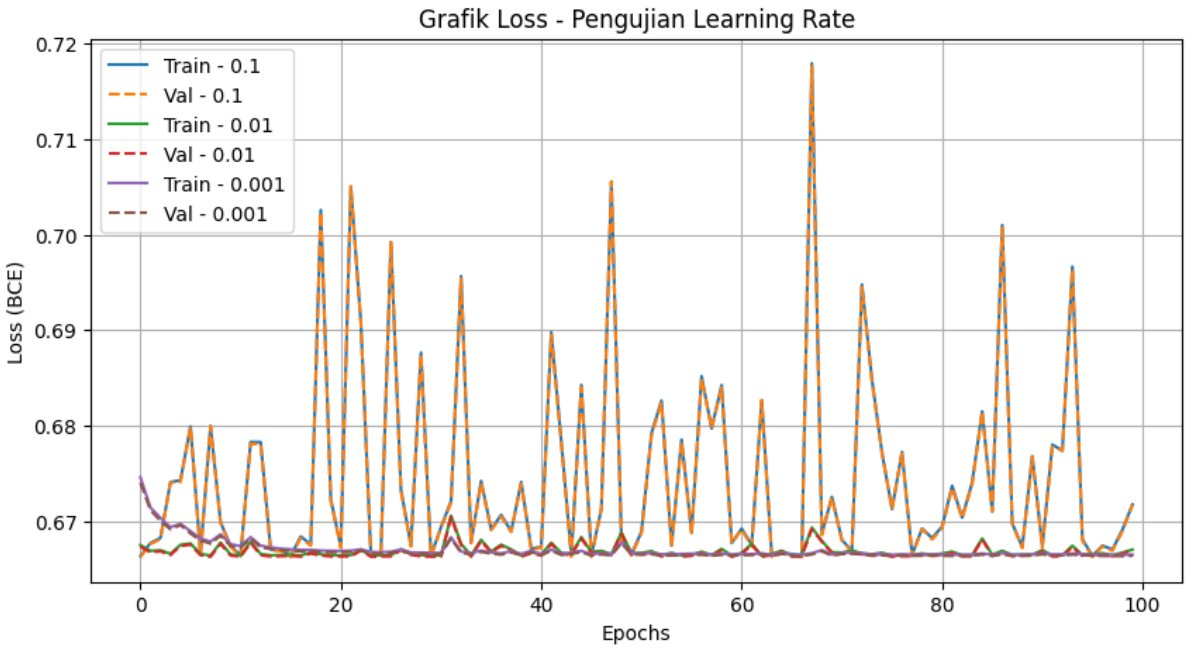
Gambar 22: Grafik *loss* untuk *learning rate* 0.1



Gambar 23: Grafik *loss* untuk *learning rate* 0.01



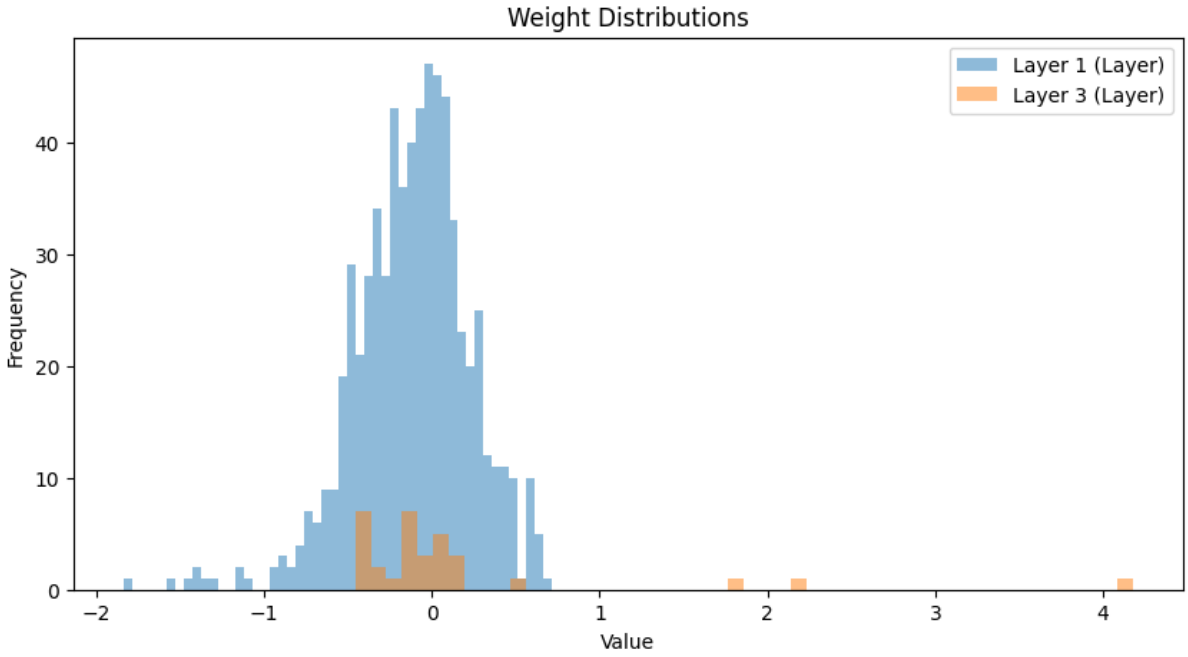
Gambar 24: Grafik *loss* untuk *learning rate* 0.001



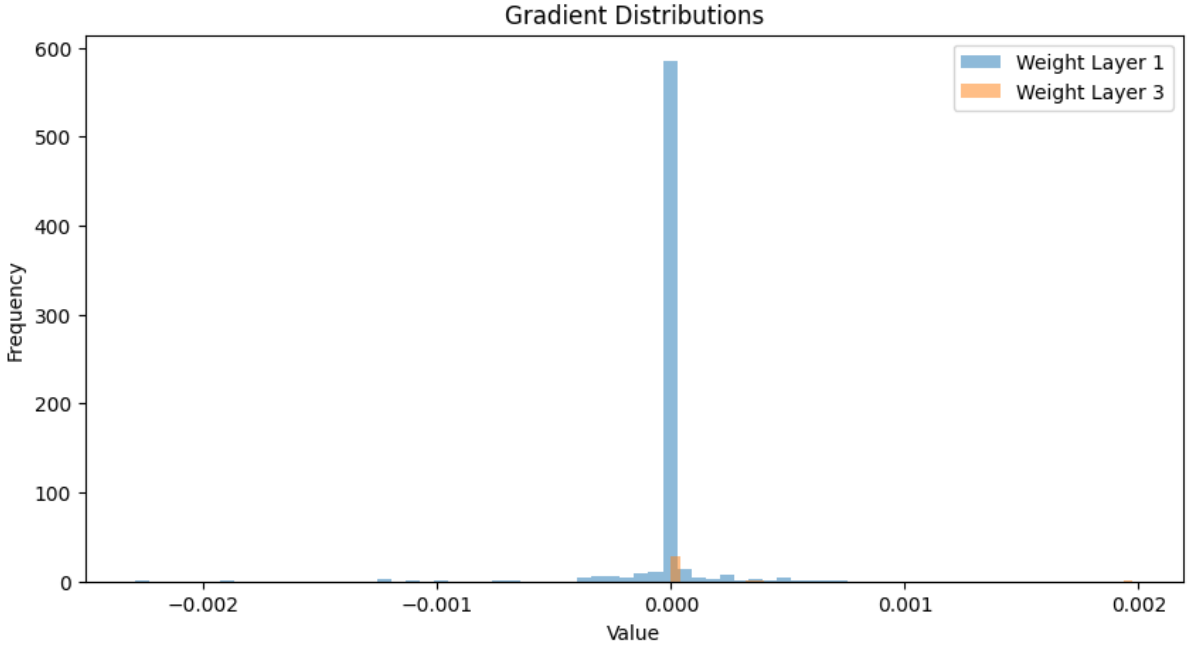
Gambar 25: Grafik *loss* gabungan ketiga *learning rate* berbeda

Berdasarkan grafik *training* dan *validation loss*, diperoleh bahwa grafik *loss* untuk tiap variasi menunjukkan perbedaan perilaku. Pada model dengan *learning rate* 0.1, grafik *loss*-nya sangat kacau dan fluktuatif. Nilai *loss* naik turun dengan ekstrem sampai mencapai nilai *loss* 0.718. Hal ini merupakan indikasi terjadinya *overshooting*, yakni ketika *step size* terlalu besar sehingga model terus-menerus melompati titik optimal dan gagal konvergen. Pada model dengan *learning rate* 0.01, pola grafiknya identik dengan pengujian *depth*, *width*, dan fungsi aktivasi sebelumnya. Grafik *loss*-nya masih tajam dan tertahan di kisaran 0.666. Hal ini menunjukkan bahwa *step size* masih terlalu besar. Pada model dengan *learning rate* 0.001, grafik *loss*-nya mulus. Nilai *loss* turun secara bertahap pada 20 epoch pertama dari 0.674 menuju 0.666, lalu mulai stabil. Fluktuasi

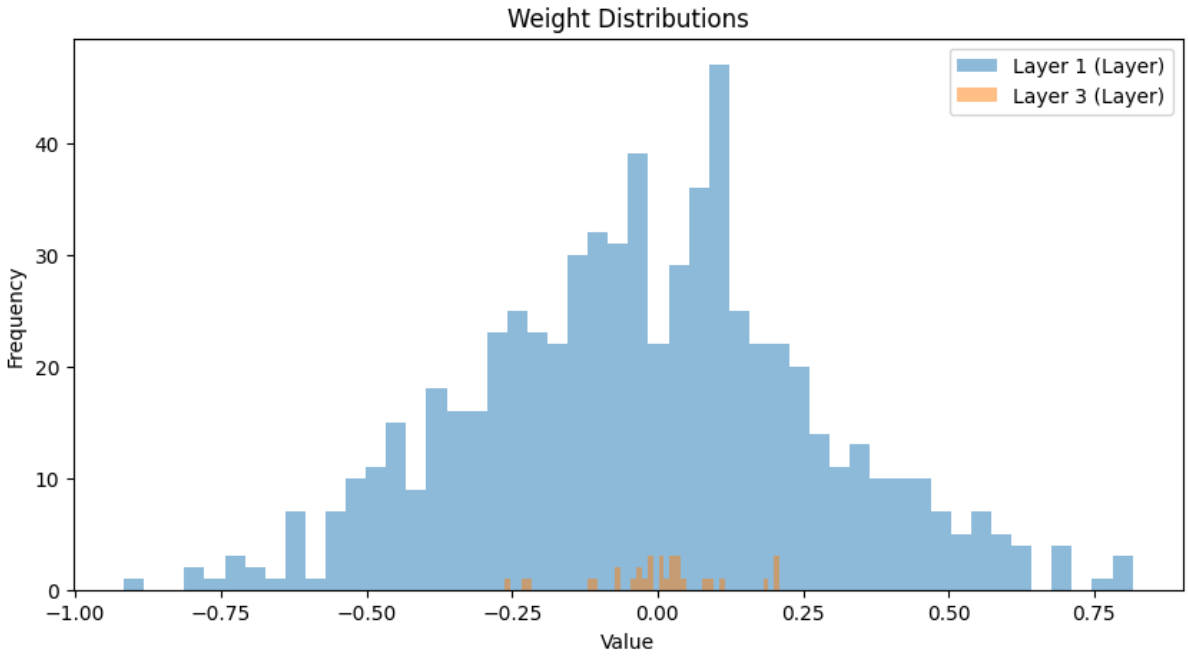
nilainya sangat kecil dan terkendali. Meskipun nilai *learning rate* 0.001 menghasilkan grafik yang jauh lebih mulus, nilai akhir *loss* di epoch 100 untuk ketiga variasi tetap berada pada $[0.666, 0.667]$.



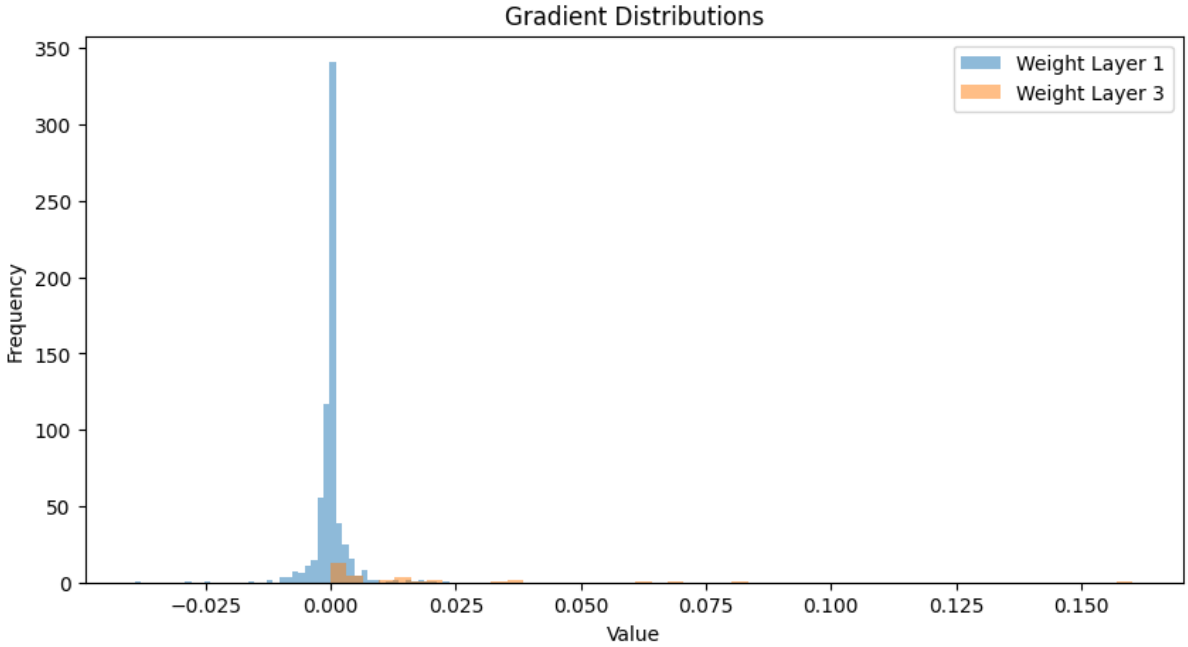
Gambar 26: Grafik distribusi bobot untuk *learning rate* 0.1



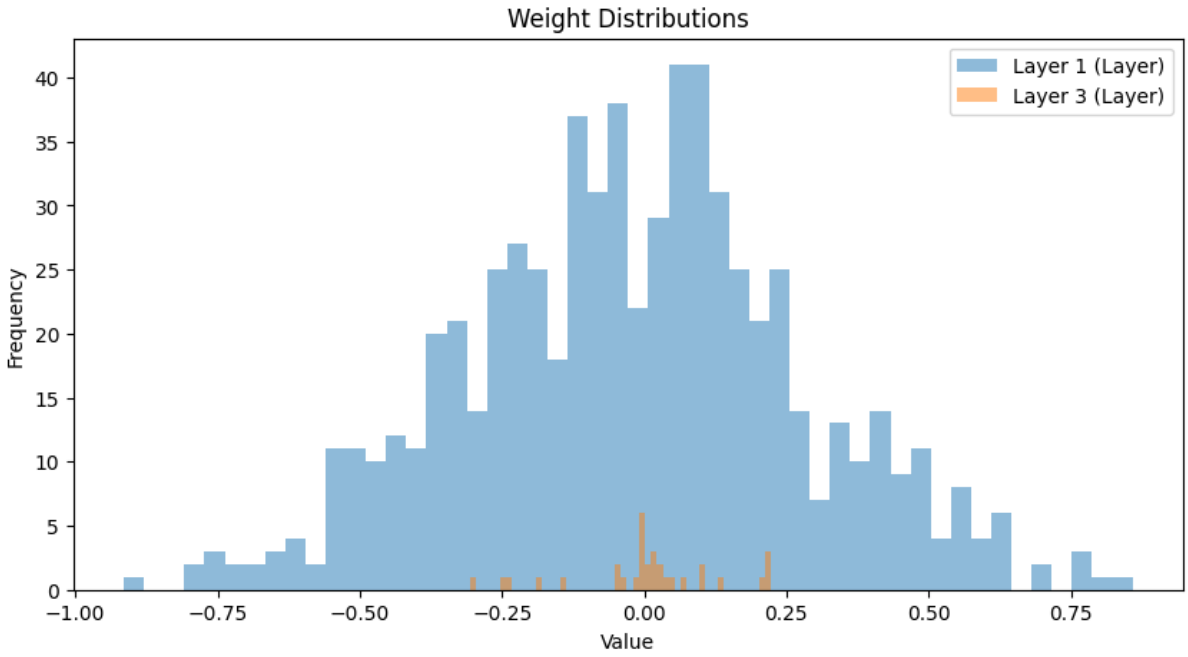
Gambar 27: Grafik distribusi gradient untuk *learning rate* 0.1



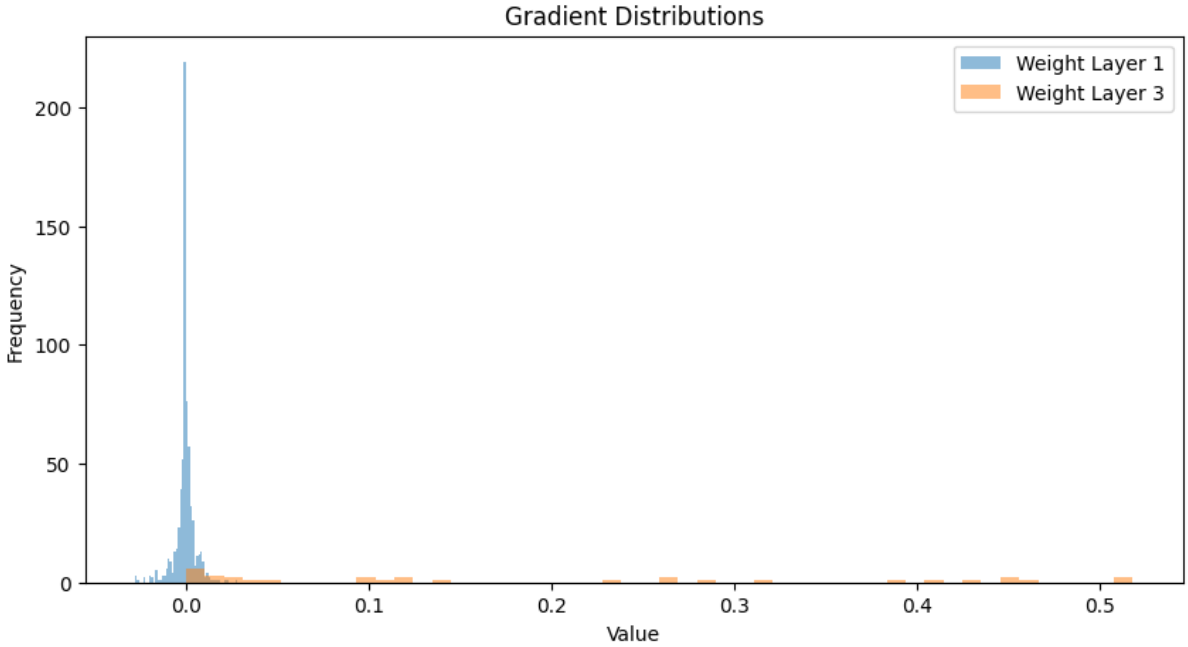
Gambar 28: Grafik distribusi bobot untuk *learning rate* 0.01



Gambar 29: Grafik distribusi gradient untuk *learning rate* 0.01



Gambar 30: Grafik distribusi bobot untuk *learning rate* 0.001



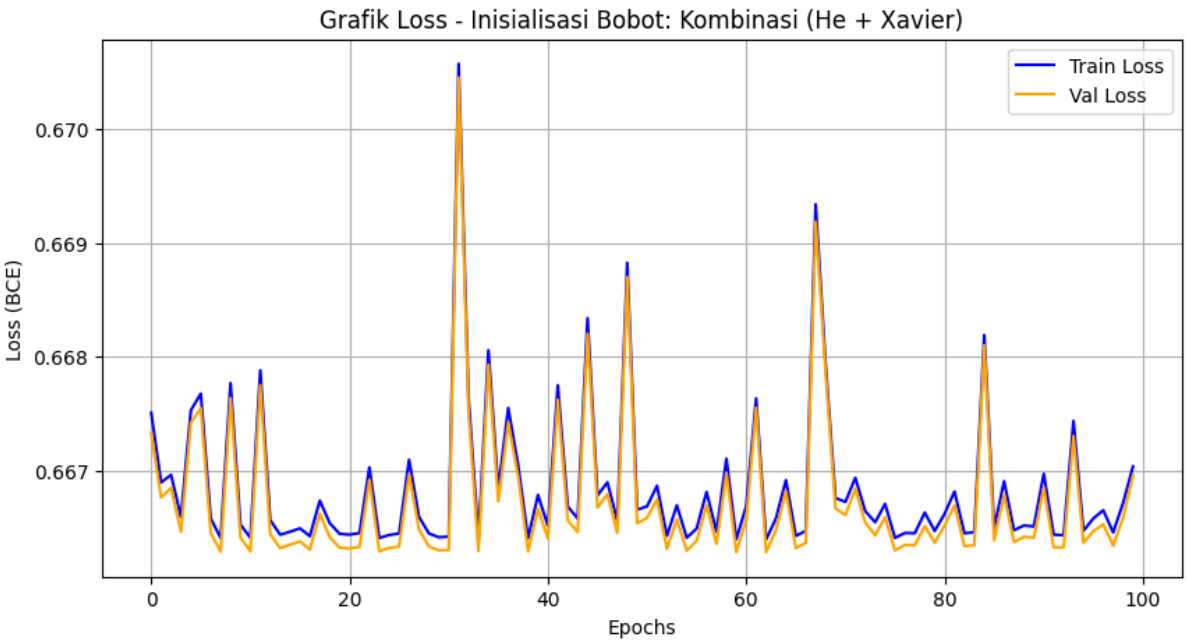
Gambar 31: Grafik distribusi gradient untuk *learning rate* 0.001

Selanjutnya, akan dilakukan analisis berdasarkan grafik distribusi bobot dan gradien. Pada model dengan *learning rate* 0.1, distribusi bobot pada *hidden layer* pertama sangat lebar, dan *output layer* memiliki nilai ekstrem, yaitu terdapat *outlier* di angka -4 dan $+4$, sehingga model menjadi tidak stabil. Distribusi gradiennya berskala sangat kecil mendekati 0. Bobot yang terlalu ekstrem membuat fungsi aktivasi seperti sigmoid menjadi jenuh, sehingga mematikan aliran gradien. Pada model dengan *learning rate* 0.01, distribusi bobotnya kembali normal, yakni berada di rentang -1 hingga 0.75, tetapi gradiennya masih sangat terpusat di angka 0. Pada model dengan *learning rate* 0.001, bobot terdistribusi seperti kurva normal. Pada *output layer*, kita bisa melihat gradiennya menyebar dengan baik hingga nilai 0.5. Hal ini menunjukkan bahwa model benar-benar

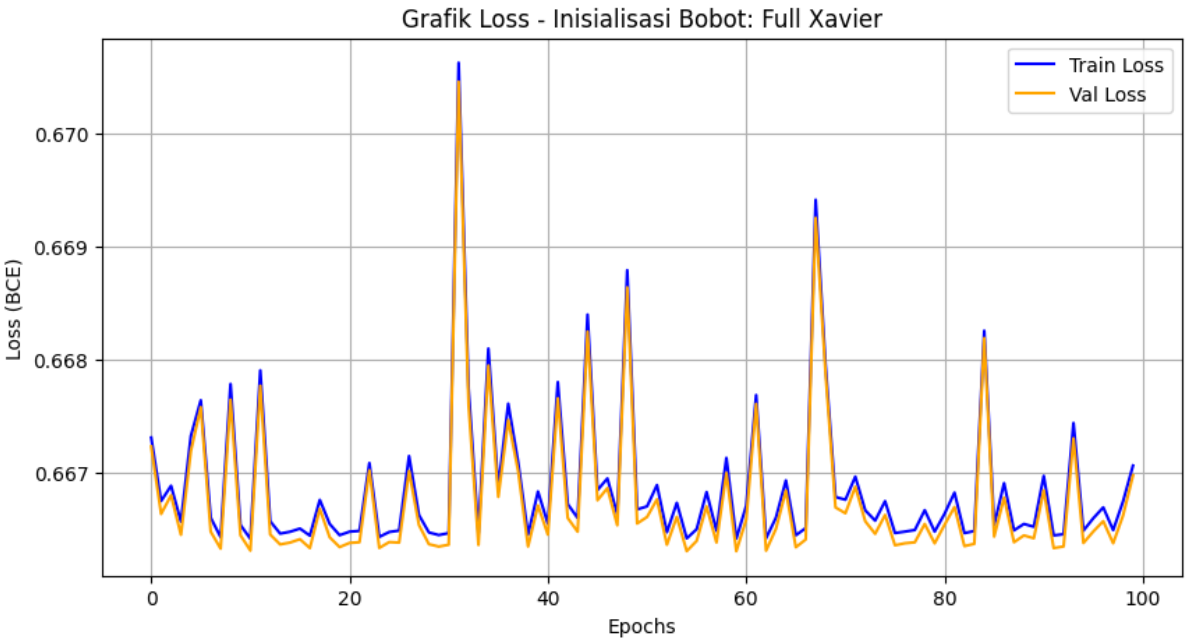
belajar dan melakukan perubahan bobot yang bermakna pada epoch awal, tidak seperti *learning rate* lebih besar yang menyebabkan aliran gradiennya mati.

2.2.4. Pengaruh Inisialisasi Bobot

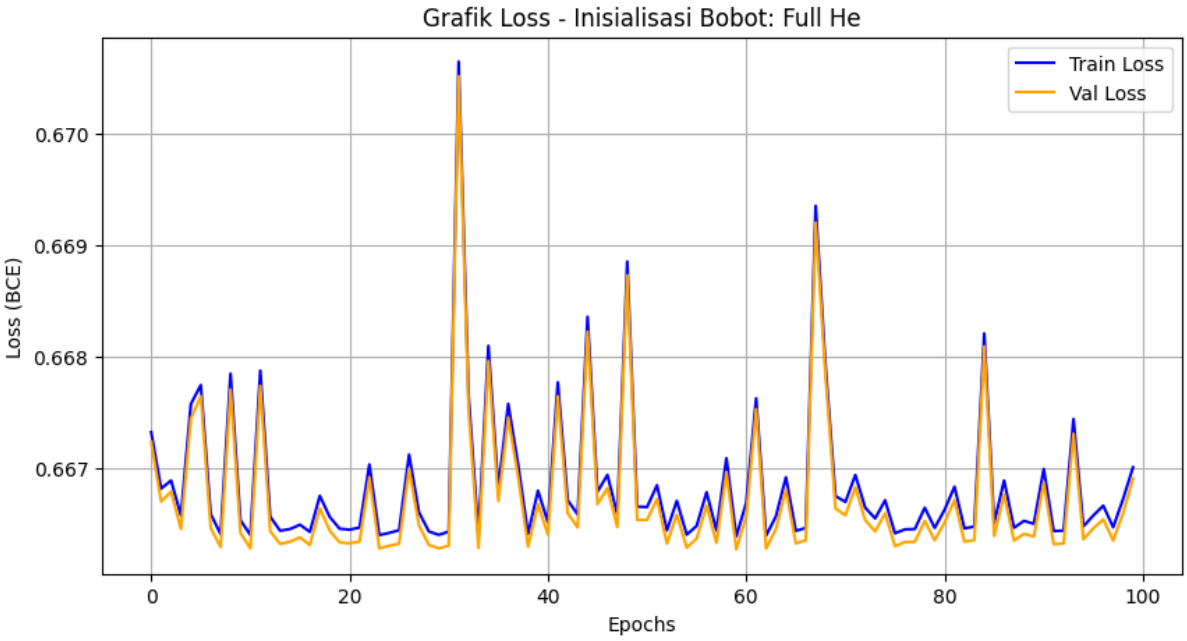
Akan ditentukan pengaruh dari inisialisasi bobot terhadap performa model. Metode inisialisasi bobot yang digunakan yaitu metode He dan metode Xavier.



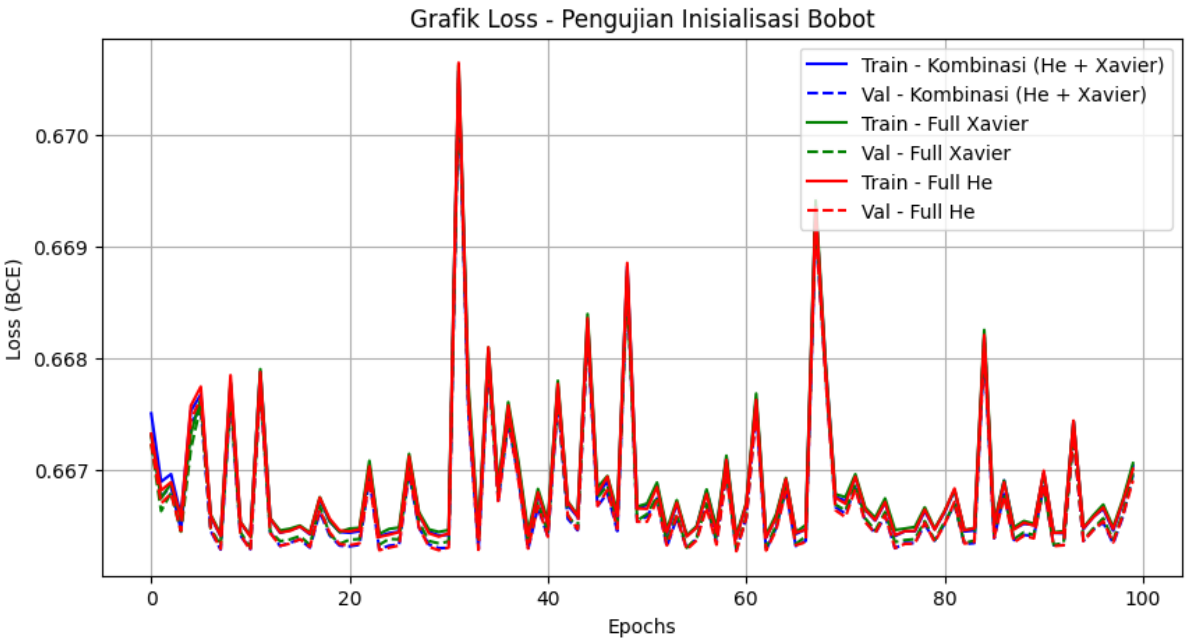
Gambar 32: Grafik *loss* untuk inisialisasi bobot He + Xavier



Gambar 33: Grafik *loss* untuk inisialisasi bobot Xavier

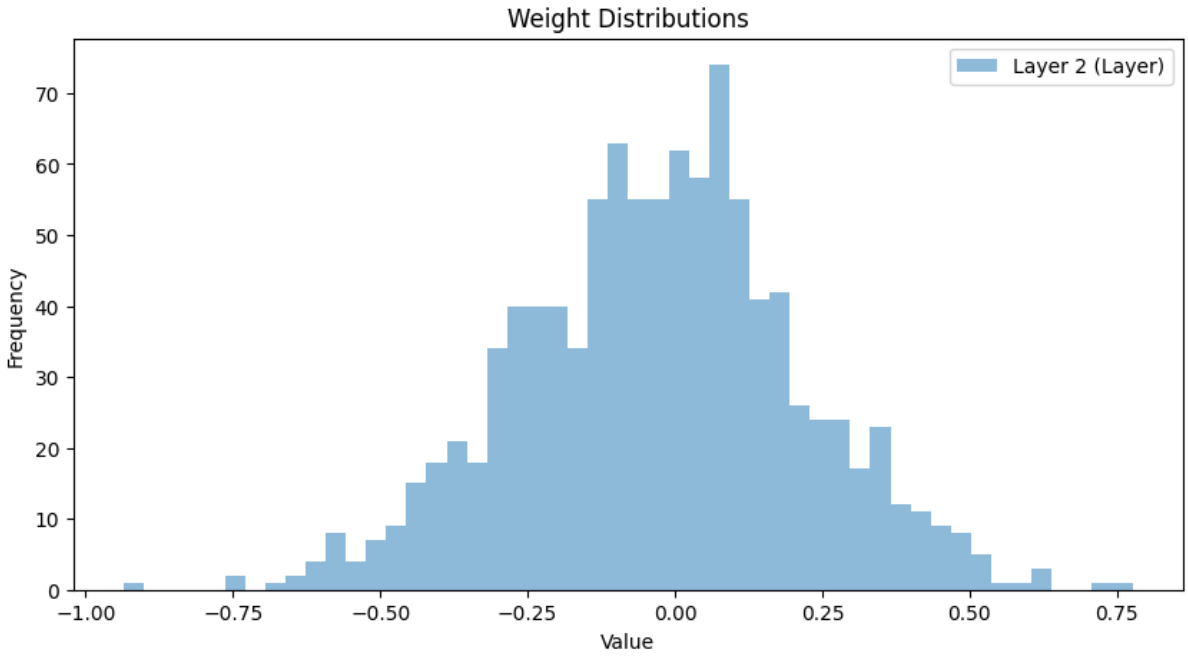


Gambar 34: Grafik *loss* untuk inisialisasi bobot He

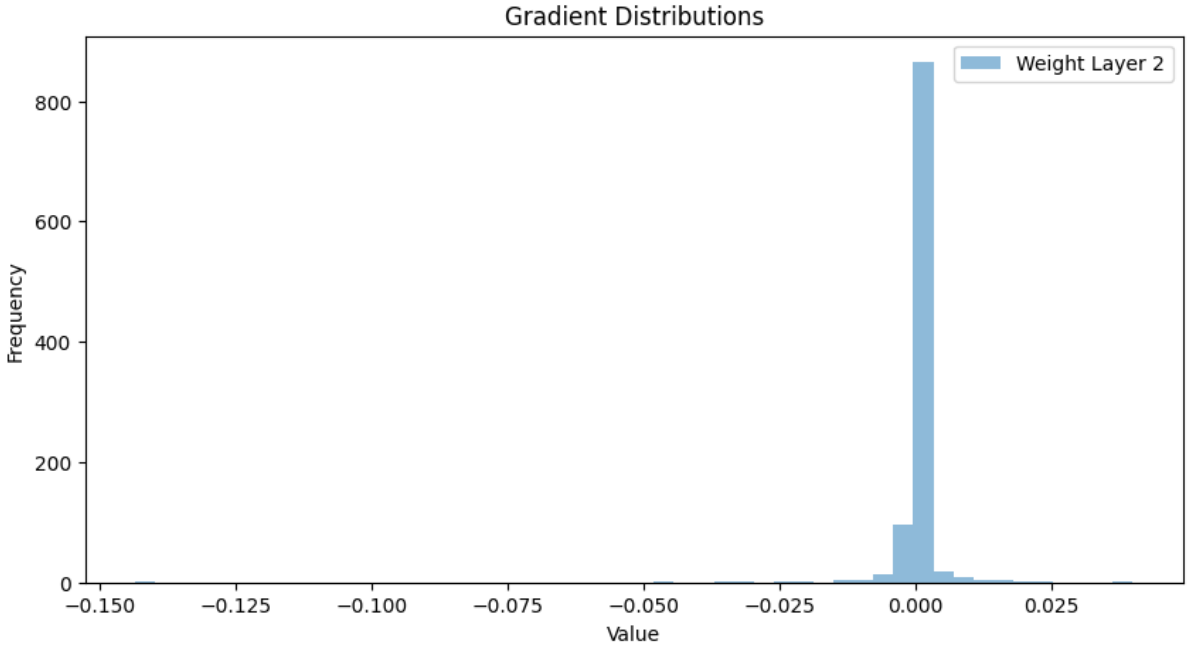


Gambar 35: Grafik *loss* gabungan ketiga variasi inisialisasi bobot

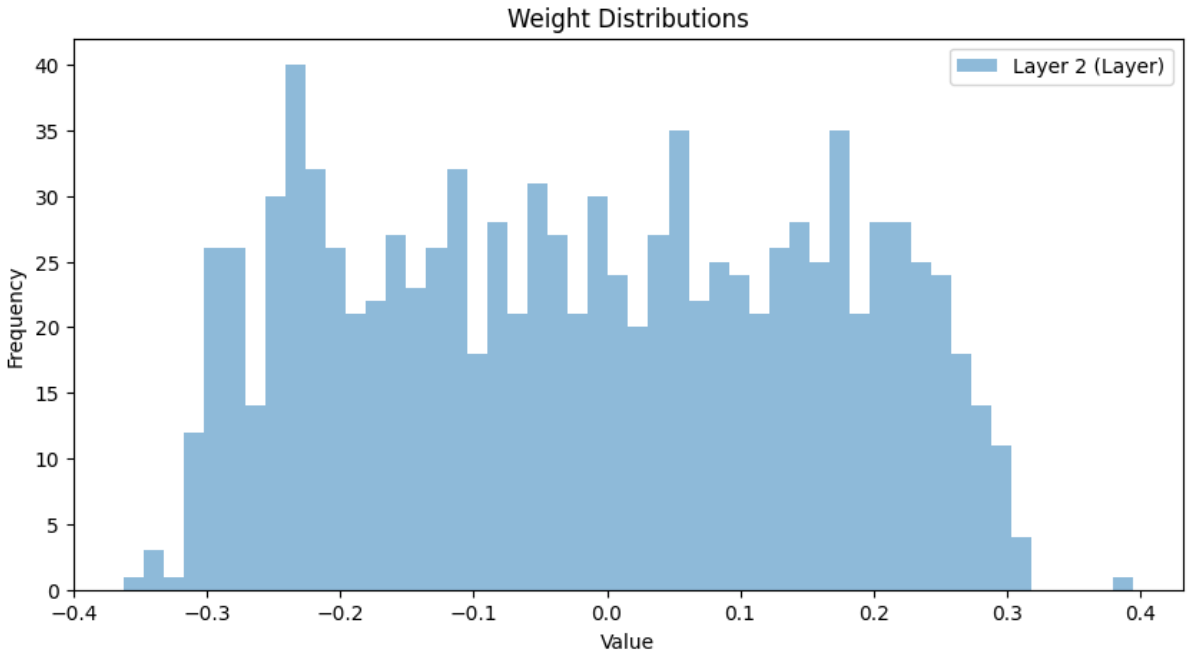
Berdasarkan grafik *training loss* dan *validation loss*, diperoleh bahwa ketiga variasi skenario inisialisasi bobot, yaitu kombinasi (He + Xavier), Xavier, dan He, menghasilkan grafik *loss* yang saling bertumpang tindih. Ketiga kurva tersebut memiliki tren fluktuasi, titik-titik lonjakan tajam seperti pada epoch 30, serta rentang nilai yang persis sama. Selain itu, nilai *loss* pada akhir pelatihan di epoch 100 untuk ketiga variasi tetap berada di angka ~ 0.667. Hal ini menunjukkan bahwa perbedaan teknik inisialisasi bobot tidak memberikan perubahan apa pun terhadap lintasan maupun nilai akhir *loss*.



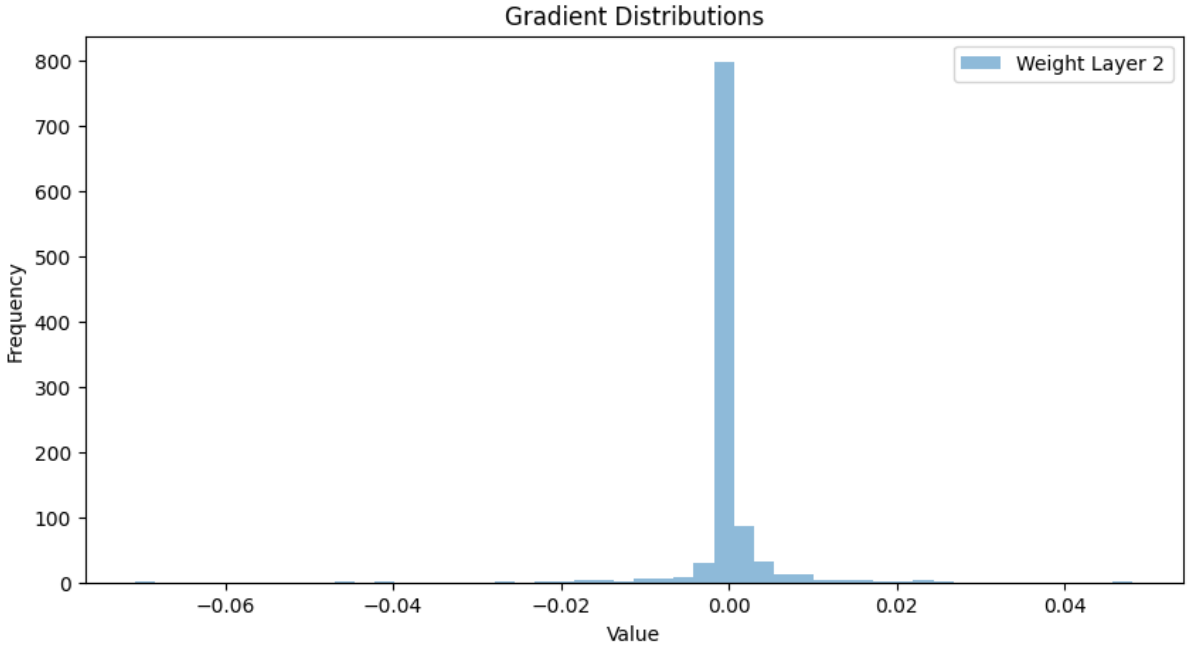
Gambar 36: Grafik distribusi bobot untuk inisialisasi bobot He + Xavier



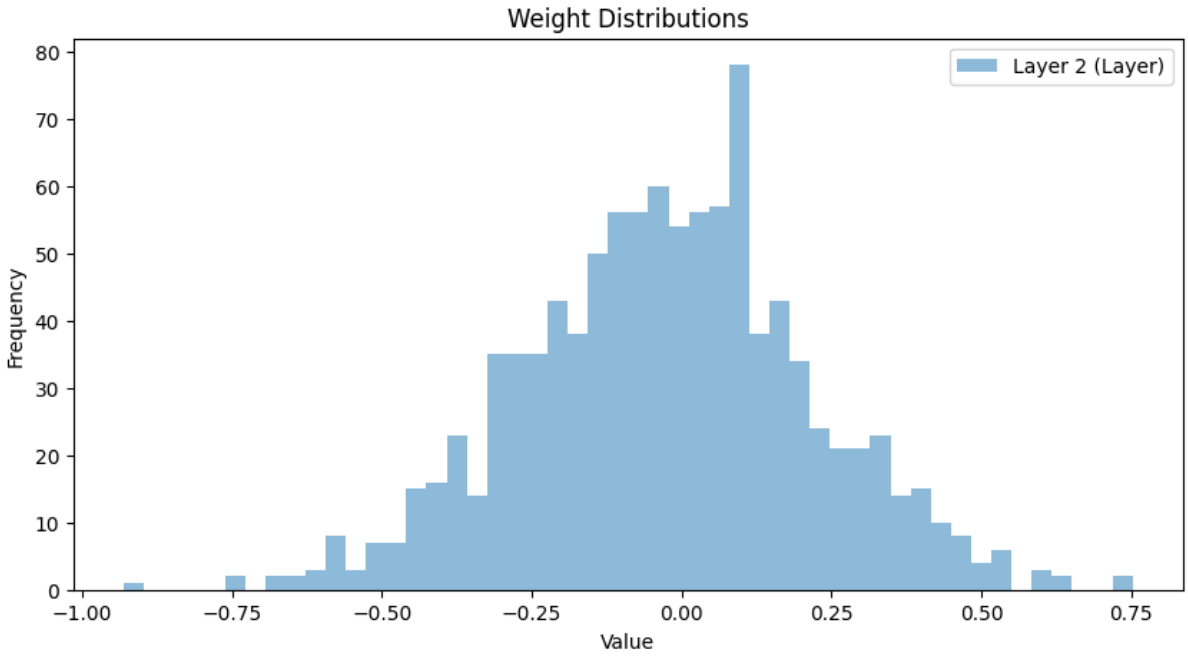
Gambar 37: Grafik distribusi gradient untuk inisialisasi bobot He + Xavier



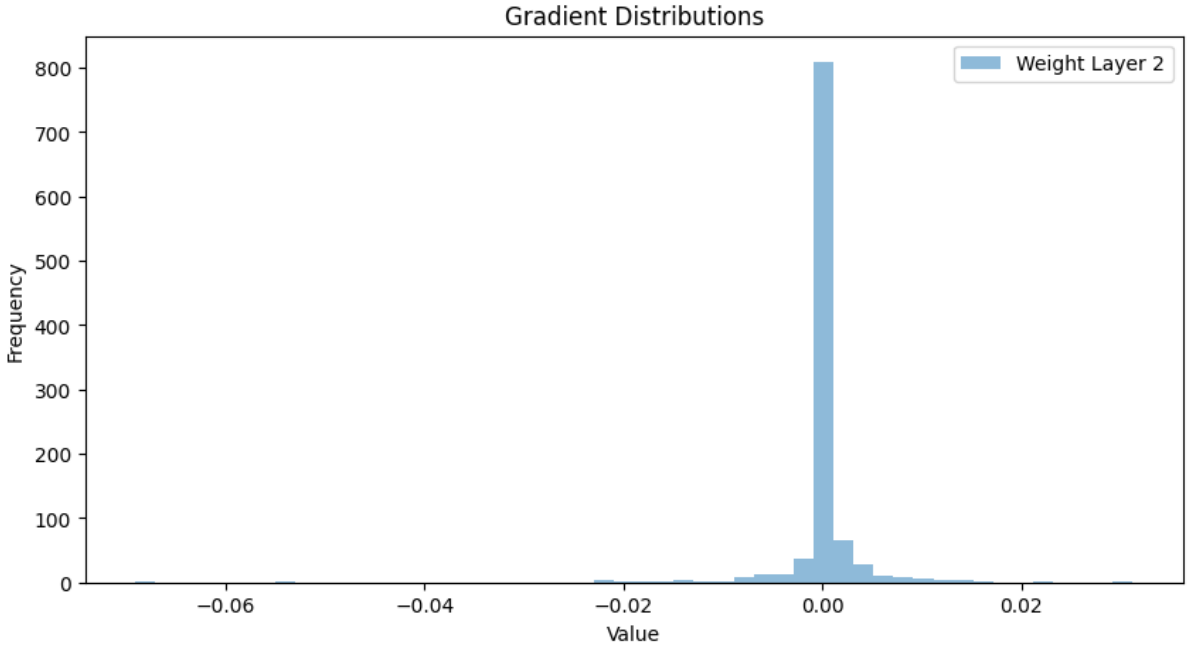
Gambar 38: Grafik distribusi bobot untuk inisialisasi bobot Xavier



Gambar 39: Grafik distribusi gradient untuk inisialisasi bobot Xavier



Gambar 40: Grafik distribusi bobot untuk inisialisasi bobot He



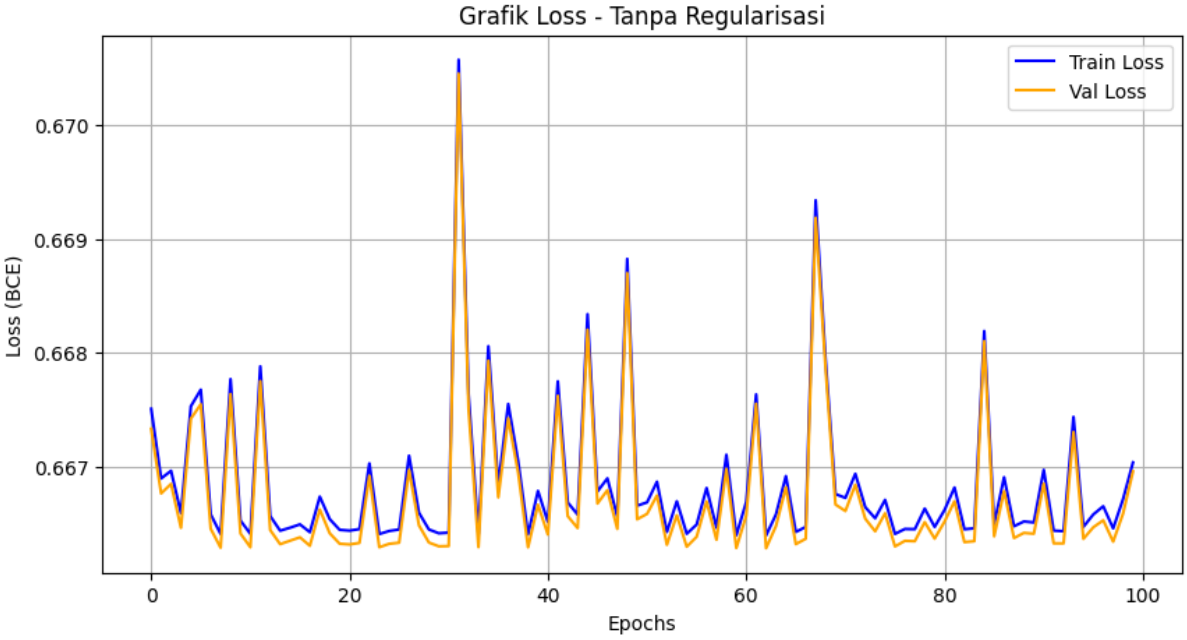
Gambar 41: Grafik distribusi gradient untuk inisialisasi bobot He

Selanjutnya, akan dilakukan analisis berdasarkan grafik distribusi bobot dan gradien. Pada grafik distribusi bobot, terlihat jelas perbedaan sebaran nilai awal parameter. Model dengan inisialisasi Xavier memiliki sebaran bobot yang lebih rapat dan menyerupai distribusi seragam di rentang sekitar -0.3 hingga 0.3. Di sisi lain, inisialisasi He menghasilkan distribusi bobot yang menyerupai kurva normal dengan rentang sebaran yang lebih lebar, yakni sekitar -0.75 hingga 0.75. Akan tetapi, grafik distribusi gradien untuk ketiga variasi menunjukkan fenomena yang identik, yakni gradien terpusat secara ekstrem di angka 0. Hal ini menunjukkan bahwa model mengalami masalah *vanishing gradient*. Meskipun inisialisasi bobot telah disesuaikan secara ideal dengan teori pasangan fungsi aktivasinya seperti ReLU dengan He dan Sigmoid dengan Xavier, metode ini tetap tidak

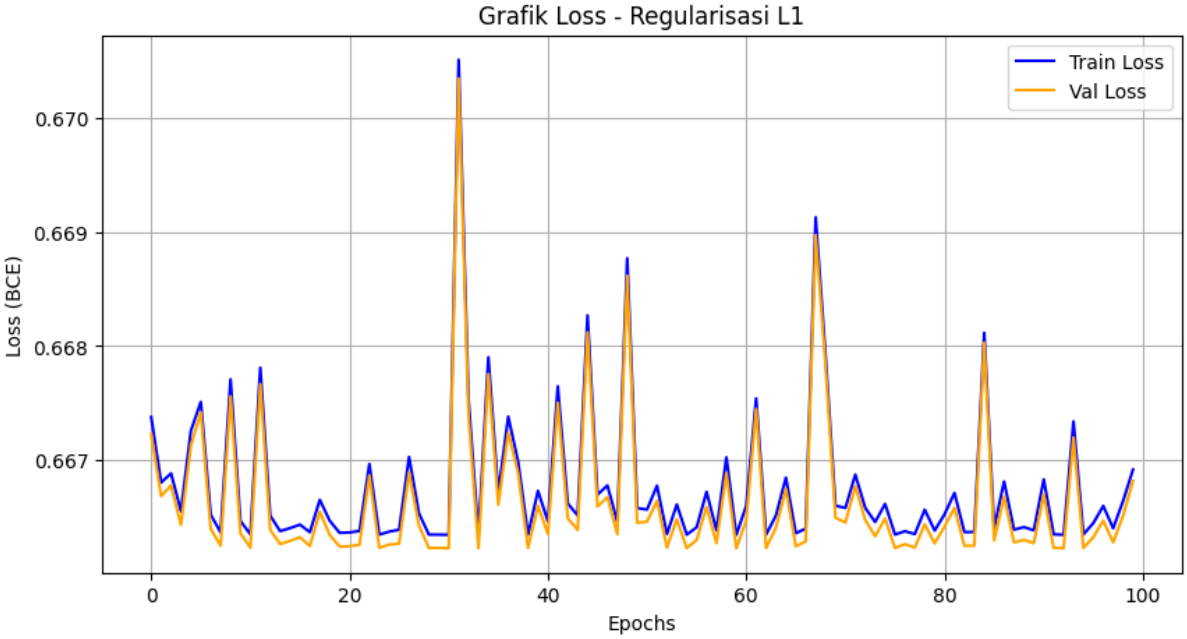
mampu memicu aliran gradien yang baik untuk melakukan pembaruan bobot. Dengan demikian, dapat disimpulkan bahwa inisialisasi bobot tidak memberikan pengaruh yang signifikan karena keterbatasan mendasar pada kapasitas model untuk mempelajari data atau terjadi *underfitting*.

2.2.5. Pengaruh Regularisasi

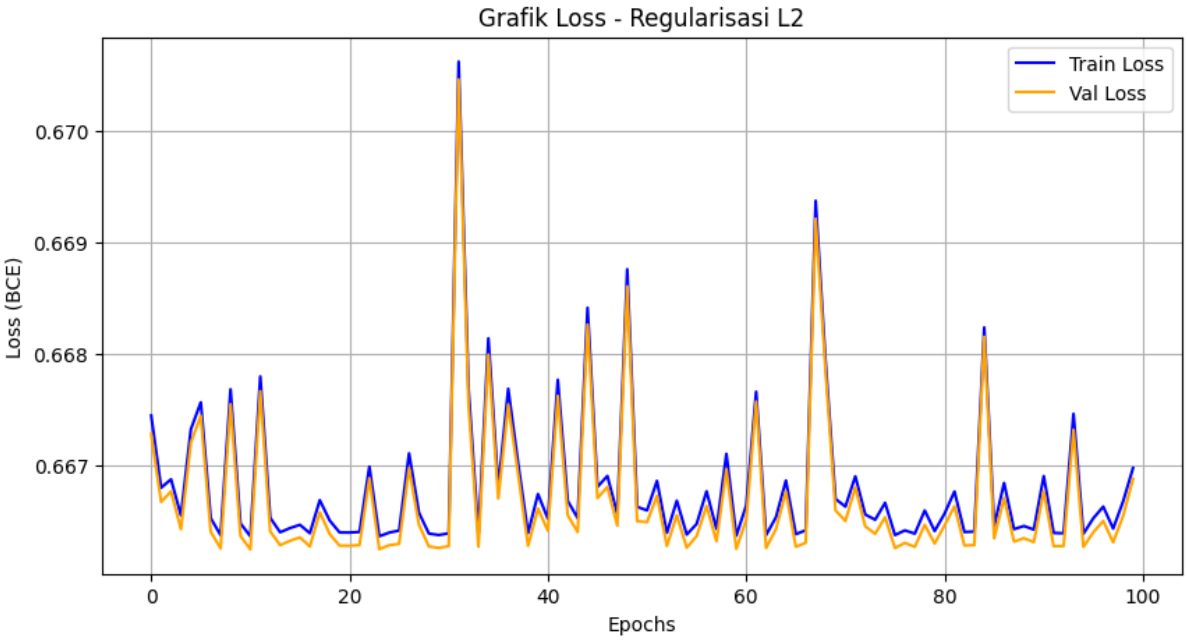
Akan ditentukan pengaruh dari penggunaan regularisasi, yakni tanpa regularisasi, dengan regularisasi $L1$, dan dengan regularisasi $L2$. Digunakan 2 *hidden layer* dengan 32 neuron dan *learning rate* sebesar 0.01 yang sama untuk ketiga pengujian. Hasil pengujian ditunjukkan pada grafik di bawah ini.



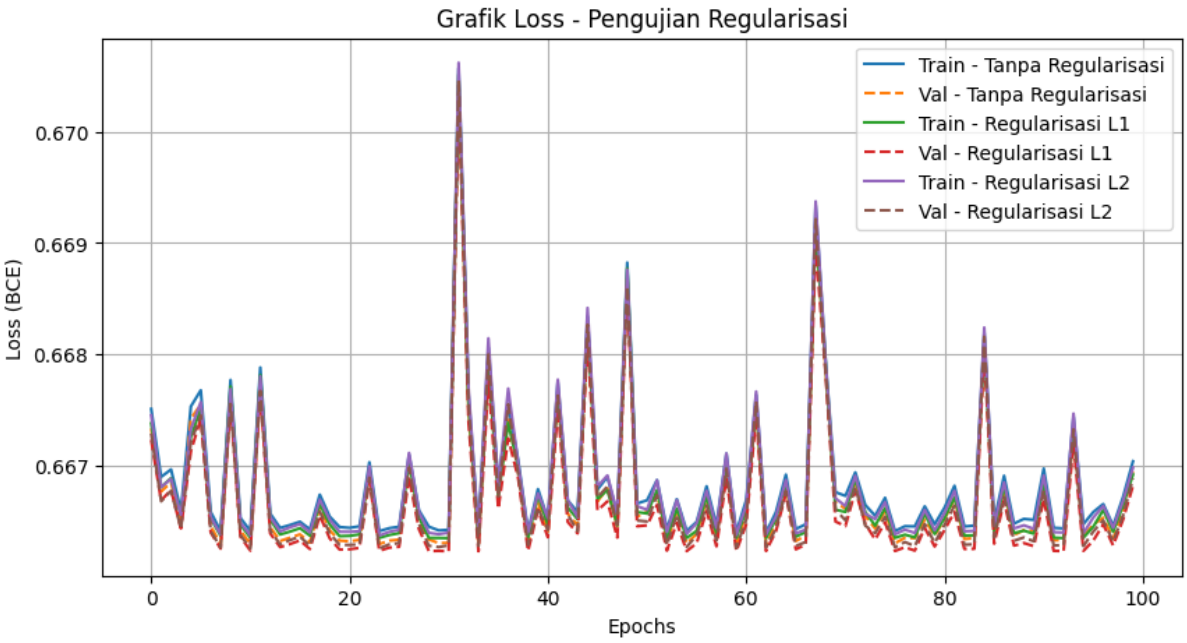
Gambar 42: Grafik *loss* untuk model tanpa regularisasi



Gambar 43: Grafik *loss* untuk model dengan regularisasi L1

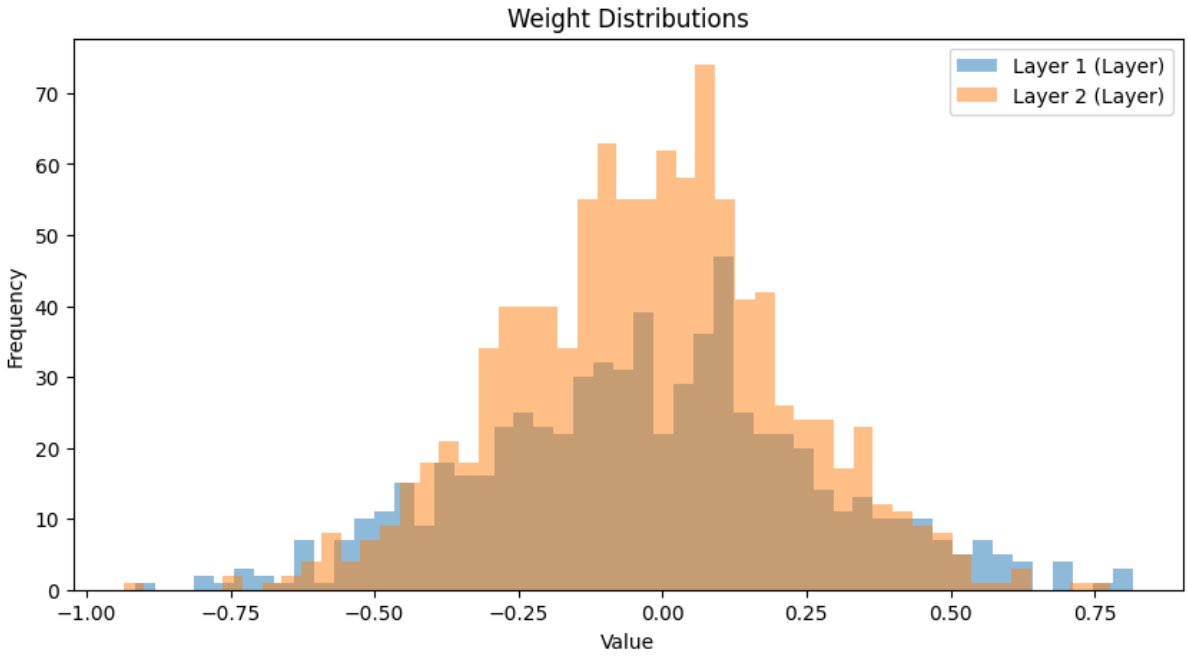


Gambar 44: Grafik *loss* untuk model dengan regularisasi L2

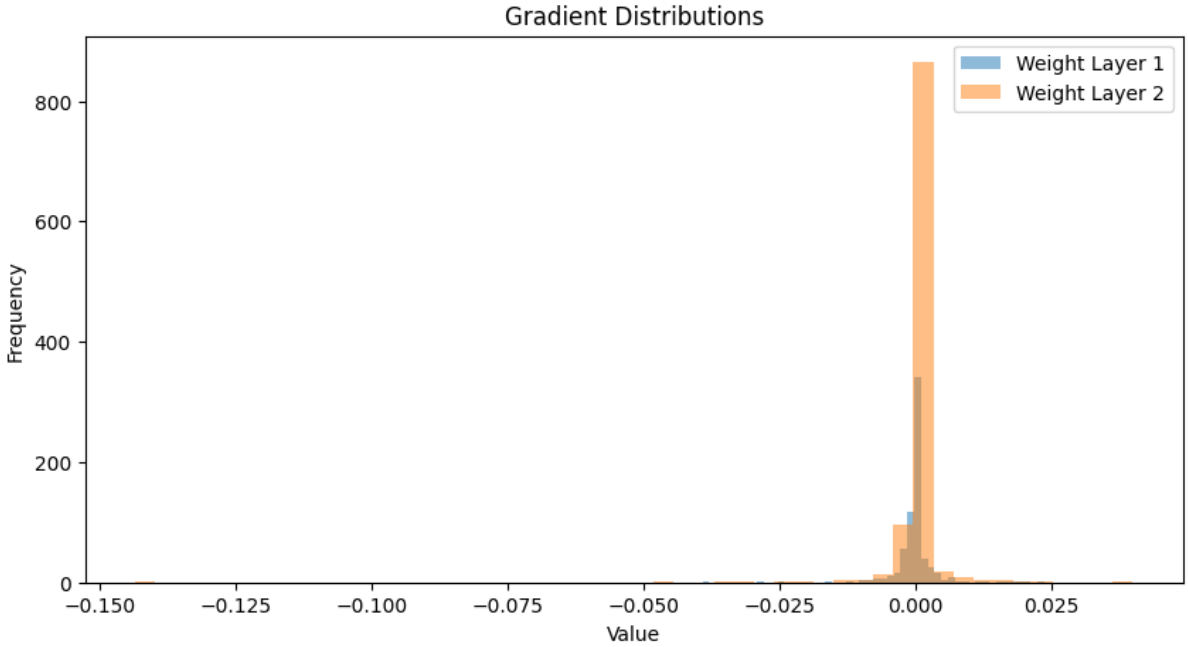


Gambar 45: Grafik *loss* gabungan untuk ketiga variasi model regularisasi

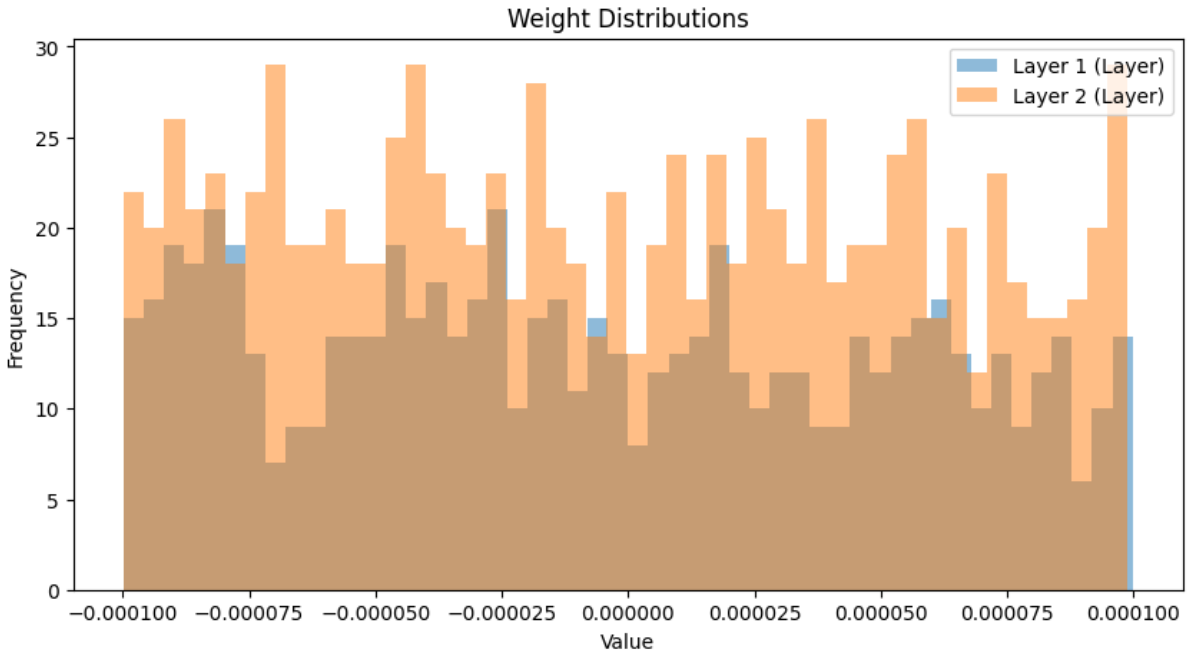
Berdasarkan grafik *training* dan *validation loss*, diperoleh bahwa ketiga variasi pengujian yang dilakukan, yakni tanpa regularisasi, dengan regularisasi $L1$, dan dengan regularisasi $L2$, menghasilkan grafik *loss* yang saling tumpang tindih dengan tren fluktuasi dan rentang nilai yang identik di sekitar $[0.666, 0.670]$. Selain itu, pada kondisi tanpa regularisasi, *validation loss* selalu bergerak rapat mengikuti *training loss*. Kondisi ini menunjukkan bahwa model tanpa regularisasi tidak mengalami *overfitting*. Oleh karena itu, penambahan regularisasi $L1$ dan $L2$ tidak memberikan dampak perubahan yang signifikan pada grafik *loss*.



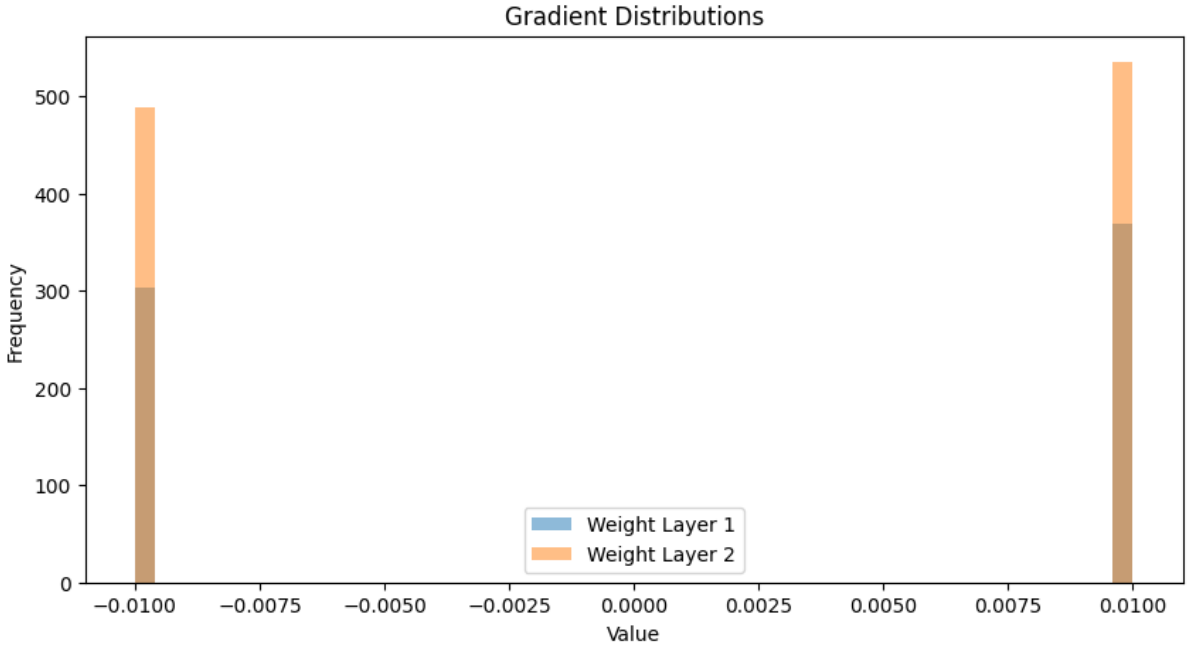
Gambar 46: Grafik distribusi bobot untuk model tanpa regularisasi



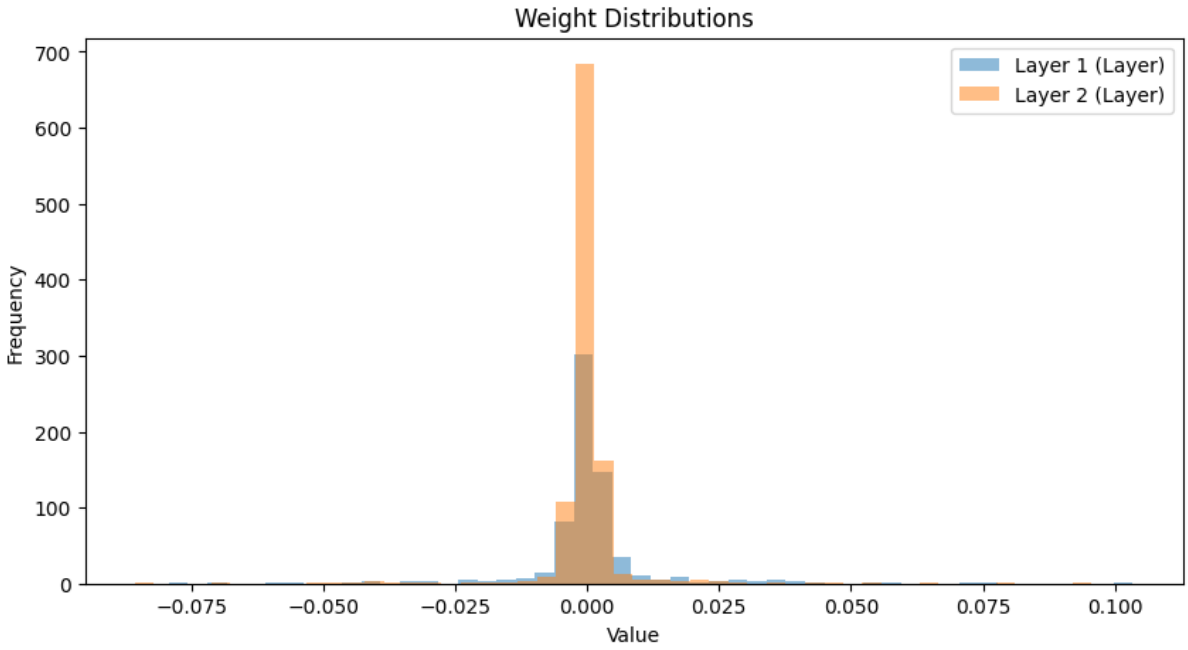
Gambar 47: Grafik distribusi gradient untuk model tanpa regularisasi



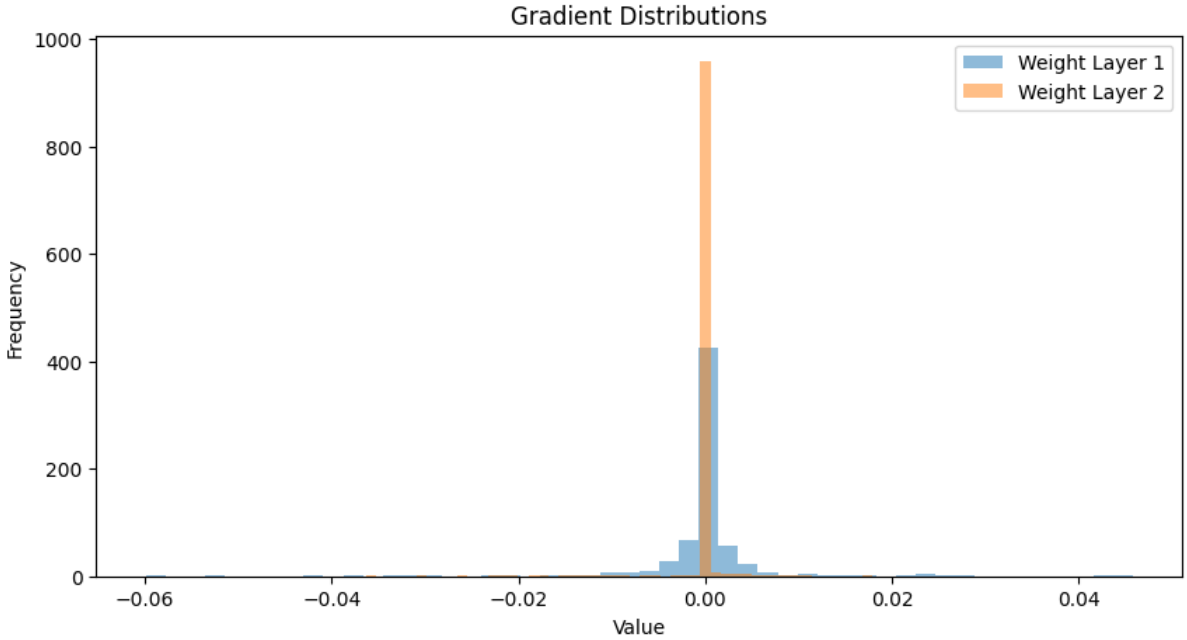
Gambar 48: Grafik distribusi bobot untuk model dengan regularisasi L1



Gambar 49: Grafik distribusi gradient untuk model dengan regularisasi L1



Gambar 50: Grafik distribusi bobot untuk model dengan regularisasi L2



Gambar 51: Grafik distribusi gradient untuk model dengan regularisasi L2

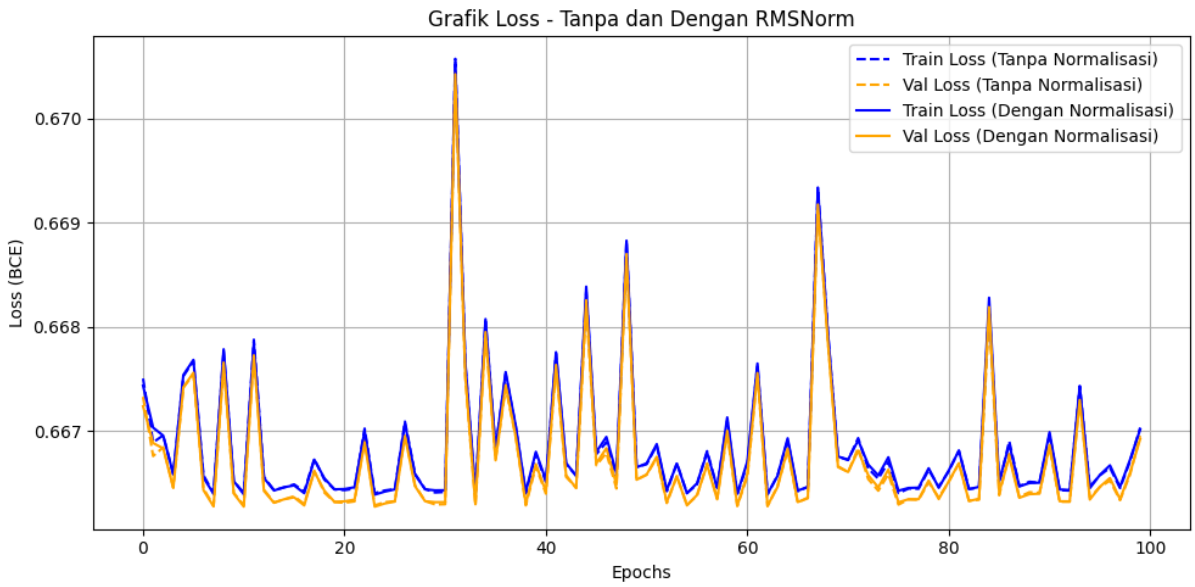
Selanjutnya, akan dilakukan analisis berdasarkan grafik distribusi bobot dan gradien. Pada model tanpa regularisasi, bobot terdistribusi dengan bentuk menyerupai kurva normal dan memiliki rentang sebaran yang paling lebar, yaitu dari -0.75 sampai dengan 0.75 serta gradien terpusat secara normal di sekitar angka 0. Pada model dengan regularisasi $L1$, skala bobot menyusut sangat ekstrem menjadi -10^{-4} hingga 10^{-4}. Hal ini akibat dari regularisasi $L1$ yang memaksa hampir keseluruhan bobot menjadi sangat dekat dengan nol. Selain itu, distribusi gradien memiliki lonjakan nilai diskrit pada dua titik, yaitu sekitar -0.01 dan 0.01. Pola ini merepresentasikan turunan dari fungsi penalti absolut regularisasi $L1$. Pada model dengan regularisasi $L2$, rentang sebaran bobot menyempit di sekitar -0.075 hingga 0.100 jika dibandingkan dengan model tanpa regularisasi, dengan

puncak kepadatan yang sangat tinggi di sekitar titik 0, serta gradien terpusat sangat tajam di sekitar 0, sesuai dengan turunan penalti regularisasi $L2$ yang berbanding lurus terhadap besaran bobotnya.

Dengan mengingat bahwa konvergensi dan lintasan grafik *loss* pada ketiga variasi pengujian tersebut hampir tidak bisa dibedakan, hasil akhir prediksi ketiganya dapat dipastikan tidak memiliki perbedaan yang signifikan. Penambahan regularisasi secara drastis mengubah struktur internal parameter model seperti distribusi bobot dan gradien, tetapi tidak mengubah nilai *loss* pada akhir proses pelatihan.

2.2.6. Pengaruh Normalisasi RMSNorm

Akan ditentukan pengaruh dari penggunaan normalisasi, yakni tanpa normalisasi dan dengan normalisasi RMSNorm. Digunakan 2 *hidden layer* dengan 32 neuron dan *learning rate* sebesar 0.01 yang sama untuk kedua pengujian. Hasil pengujian ditunjukkan pada grafik di bawah ini.



Gambar 52: Grafik *loss* gabungan model tanpa normalisasi dan model normalisasi dengan RMSNorm

Berdasarkan grafik *training* dan *validation loss*, diperoleh bahwa kurva *training loss* dan *validation loss* dari model dengan RMSNorm bertumpang tindih dengan model tanpa normalisasi sepanjang 100 epoch. Keduanya menunjukkan pola fluktuasi yang serupa, termasuk titik-titik lonjakan *loss* yang tajam, seperti pada epoch 30, 48, dan 68. *Validation loss* secara konsisten mengikuti pergerakan *training loss* dengan jarak yang rapat pada kedua variasi. Hal ini menunjukkan bahwa penambahan normalisasi dengan RMSNorm tidak mengubah proses pelatihan secara signifikan.

2.2.7. Perbandingan dengan *Library* Sklearn

Akan ditentukan perbandingan dari penggunaan model *Multi-Layer Perceptron* (MLP) dari *library* yang dibuat oleh kami dengan *library* dari Scikit-learn. Digunakan 2 *hidden layer* dengan 32 neuron dan *learning rate* sebesar 0.01 yang sama untuk kedua pengujian dengan *library* MLP yang berbeda. Hasil pengujian ditunjukkan pada tabel di bawah ini.

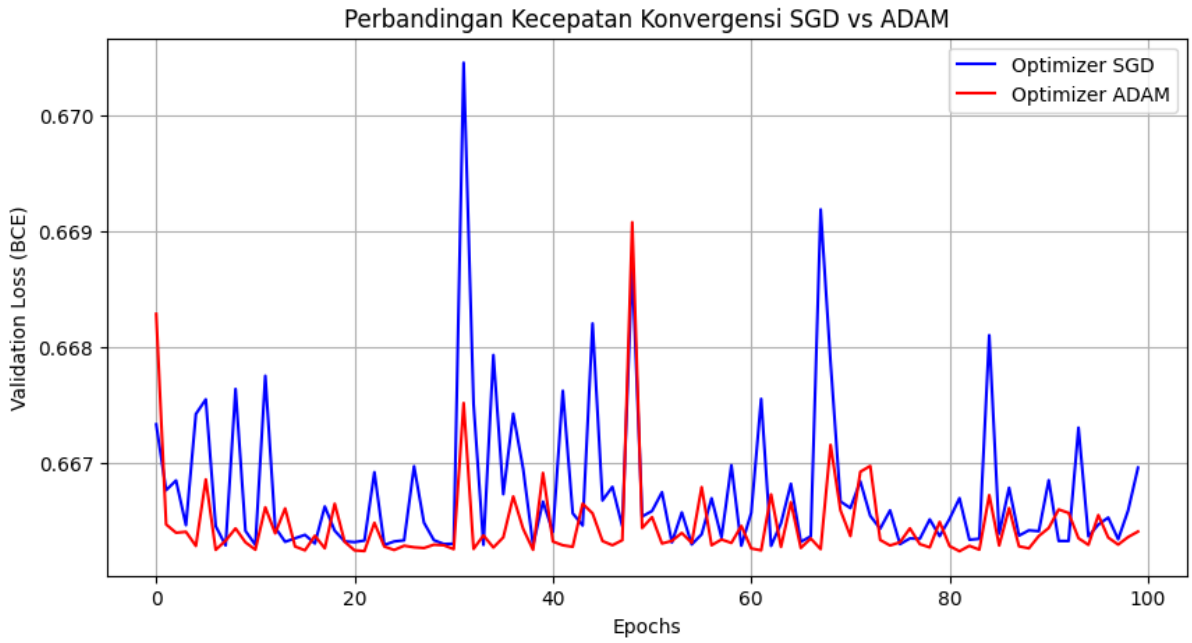
Tabel 19: Hasil pengujian kedua *library* pada epoch 100

Library	***Training Loss***	***Validation Loss***
Six-seven	0.667	0.667
Sklearn	0.417	0.596
Persentase selisih	25.03%	7.13%

Berdasarkan tabel di atas, terlihat bahwa implementasi dari *library* Sklearn terbukti lebih baik dalam meminimalkan kesalahan selama fase pelatihan. Sklearn menekan *training loss* hingga 0.417, sedangkan *library* buatan sendiri (Six-seven) tertahan pada angka 0.667. Selisih 25.03% menunjukkan perbedaan *library* yang signifikan. Selain itu, Sklearn memiliki kemampuan generalisasi prediksi yang lebih baik pada data yang tidak terlihat. Hal ini terlihat dari nilai *validation loss* sebesar 0.596 dibandingkan *library* Six-seven sebesar 0.667, dengan selisih performa 7.13%. *Library* Six-seven menghasilkan nilai *training* dan *validation loss* yang serupa, yaitu sebesar 0.667. Kondisi statis ini menunjukkan bahwa model gagal mempelajari pola data secara efektif. Model kemungkinan terjebak pada minimum lokal, mengalami masalah *vanishing gradient*, atau kalah saing dengan mekanisme inisialisasi dan optimasi bobot internal bawaan Sklearn yang lebih terkalibrasi.

2.2.8. Perbandingan *Optimizer* SGD vs ADAM

Akan ditentukan perbandingan dari penggunaan *optimizer* yang berbeda, yakni *Stochastic Gradient Descent* (SGD) dan *Adaptive Moment Estimation* (ADAM). Digunakan 2 *hidden layer* dengan 32 neuron dan *learning rate* sebesar 0.01 yang sama untuk kedua pengujian dengan *optimizer* yang berbeda. Hasil pengujian ditunjukkan pada grafik di bawah ini.



Gambar 53: Grafik *loss* gabungan *Optimizer* SGD vs ADAM

Berdasarkan grafik *training loss* dan *validation loss*, diperoleh bahwa *optimizer* ADAM (garis merah) menghasilkan pergerakan yang lebih stabil dibandingkan *optimizer* SGD (garis biru). *Optimizer* SGD mengalami fluktuasi yang sangat ekstrem dengan beberapa lonjakan *loss* tinggi seperti pada epoch 30 dan 67, sedangkan fluktuasi *optimizer* ADAM jauh lebih teredam dan bergerak di rentang nilai yang lebih sempit. Secara keseluruhan, lintasan *validation loss optimizer* ADAM konsisten berada di bawah *optimizer* SGD. *Optimizer* ADAM lebih efektif dalam menekan nilai *loss* mendekati batas bawah grafik sekitar 0.666 dan mempertahankan posisi tersebut hampir di sepanjang pelatihan sampai dengan 100 epoch. Sebaliknya, *optimizer* SGD terus berosilasi dan sering kali gagal mempertahankan nilai *loss* terendahnya.

BAB III
KESIMPULAN DAN SARAN

3.1. Kesimpulan

Sebagai kesimpulan, melalui Tugas Besar 1 ini, kami dapat mengimplementasikan suatu *Feed-Forward Neural Nework* (FFNN) atau *Multi-Layer Perceptron* (MLP) dari *scratch* hanya dengan `Python` dan `numpy`. Berdasarkan implementasi *Feedforward Neural Netrwork* dari Sctratch dan pengujian hyperparameter, regularisasi, serta perbandingan dengan *library* Sklearn, diperoleh kesimpulan sebagai berikut.

1. **Implementasi FFNN**. Implementasi berupa package yang mudah digunakan, tetapi *customizable* sesuai spesifikasi. Fitur utama dari implementasi adalah *automatic differentiation* untuk *forward* dan *backward pass*, fungsi aktivasi dan *loss* custom, metode optimisasi yang modular, dan konstruktor `MLP` dan `Layer` yang intuitif.

2. **Pengaruh *Depth* dan *Width***. Perubahan *depth* (1 hingga 3 layer) dan *width* (16 hingga 64 neuron) tidak memberikan peningkatan performa yang signifikan. Model mengalami *underfitting* dan terjebak di titik minimum lokal dengan nilai akhir *loss* stagnan di kisaran 0.667.

3. **Pengaruh Fungsi Aktivasi**. Variasi fungsi aktivasi seperti ReLU, Sigmoid, Tanh, dan GeLU, menghasilkan pola *loss* yang serupa. Peninjauan distribusi gradien menunjukkan bahwa mayoritas nilai terpusat di angka 0, menunjukkan terjadinya masalah *vanishing gradient*, sehingga model berhenti melakukan pembaruan bobot secara bermakna.

4. **Pengaruh *Learning Rate***. Nilai l*earning rate* 0.1 menyebabkan *overshooting*, sedangkan nilai 0.01 masih terlalu besar dan membuat model stagnan. Meskipun nilai 0.001 memberikan konvergensi yang lebih mulus dan distribusi gradien yang lebih menyebar di awal epoch, nilai akhir *loss* tetap tidak mampu turun dari angka 0.667.

5. **Pengaruh Inisialisasi Bobot**. Penggunaan inisialisasi He dan Xavier berhasil membentuk sebaran parameter awal sesuai dengan teorinya, tetapi gagal mengatasi masalah *vanishing gradient*. Hal ini menunjukkan bahwa stagnasi performa model disebabkan oleh keterbatasan mendasar pada arsitektur dalam memetakan data, bukan sekadar pada teknik inisialisasi.

6. **Pengaruh Regularisasi.** Karena model dasar tidak mengalami *overfitting* sejak awal, penambahan regularisasi L1 maupun L2 mengubah distribusi bobot secara drastis, tetapi tidak memberikan perbaikan apa pun pada hasil akhir prediksi.

7. **Pengaruh Normalisasi RMSNorm**. Implementasi RMSNorm menghasilkan kurva *loss* yang bertumpang tindih dengan model tanpa normalisasi, menunjukkan bahwa penambahan *layer* normalisasi ini tidak memengaruhi atau mempercepat proses pelatihan secara signifikan.

8. **Perbandingan dengan Library Sklearn**. Fungsionalitas bawaan Scikit-learn lebih superior dibandingkan implementasi model dari scratch (Six-seven) dengan konfigurasi hyperparameter yang sama persis. Sklearn berhasil menekan *training loss* hingga 0.417 dan *validation loss* hingga 0.596, dibandingkan Six-seven yang bernilai 0.667.

9. **Perbandingan Optimizer SGD vs ADAM**. Optimizer ADAM terbukti lebih unggul karena mampu menghasilkan konvergensi yang lebih stabil, minim fluktuasi ekstrem, dan secara konsisten mempertahankan nilai *loss* terendah dibandingkan algoritma SGD yang sangat fluktuatif.

3.2. Saran

Dari segi pembangunan package FFNN, beberapa saran yang bisa kami berikan melalui pengalaman implementasinya adalah sebagai berikut.

1. Modularisasi yang lebih bersih. Terkadang tidak konsisten apakah implementasi suatu fitur perlu kelas baru, fungsi saja, atau lainnya. Mungkin lebih baik jika arsiektur packagenya sendiri juga dipikirkan dengan lebih matang.

2. Implementasi lebih banyak fungsi aktivasi, fungsi *loss*, dan metode optimisasi untuk *customizability* dan eksperimen. Contohnya adalah metode *momentum* atau *hinge loss*.

3. Implementasi *automaic differentiation* melalui kelas `Tensor` dan operasi-operasi dasarnya sangat efisien, dan sebaiknya diimplementasikan selalu untuk semua *development* library *neural network* (bahkan terlepas bonus dari tugas besar ini, karena membuat semuanya menjadi lebih mudah).

LAMPIRAN

3.3. Lampiran Tautan

Tabel 20: Tautan

Nama	**Link**
GitHub Repository	https://github.com/timoruslim/Tubes1_Sixseven

3.4. Lampiran Pembagian Tugas

Tabel 21: Pembagian Tugas

NIM	**Tugas**
10122062	analisis dan pemodelan dataset, laporan
10123053	implementasi FFNN dari *scratch*, laporan

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	:	Albi Arrizkya Putra (10122062),
		Timothy Niels Ruslim (10123053)
Fakultas/Sekolah	:	Sekolah Teknik Elektro dan Informatika (STEI)
Kode Mata Kuliah	:	IF3270
Nama Mata Kuliah	:	Pembelajaran Mesin
Judul Tugas/Proposal	:	Tugas Besar 1: Feedforward Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: tidak berlaku.	Tidak	1
Pembuatan Teks: menggunakan *tools* seperti Gemini.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan *tools* seperti Gemini.	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: tidak berlaku.	Tidak	1
Pembuatan Kode: menggunakan *tools* seperti Gemini dan Copilot.	Ya	3

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Maret 2026

Timothy Niels Ruslim

10123053

REFERENSI

[1] Alaa El Alaoui. On the GELU activation function. https://alaaalatif.github.io/2019-04-11-gelu/, 2019. Accessed: 2026-03-16.

[2] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256. PMLR, 2010.

[3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *arXiv preprint*, 2015.

[4] Andrej Karpathy. The spelled-out intro to neural networks and backpropagation: building micrograd. YouTube video, 2022.

[5] Rohan Mehta. Automatic differentiation in neural nets. https://mehta-rohan.com/writings/blog_posts/autodiff.html, 2021. Accessed: 2026-03-16.

[6] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. Searching for activation functions. *arXiv preprint*, 2017.

[7] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019.